

基本情報技術者試験 科目 B アルゴリズム対策

目次

基本情報技術者試験 科目 B アルゴリズム対策	1
1. アルゴリズム問題の概要と勉強法	2
1.1. アルゴリズム問題の概要	2
1.2. アルゴリズム問題の対策	6
2. 基本アルゴリズム.....	8
2.1. 基本アルゴリズムとは	8
2.2. データの持ち方（データ構造）	9
2.3. データ探索アルゴリズム.....	16
2.4. データ整列アルゴリズム.....	18
3. 疑似言語の文法	22
3.1. 疑似言語の原則.....	22
3.2. 疑似言語の文法.....	23
3.3. 疑似言語の読み方確認	30
3.4. 疑似言語のトレース.....	32
4. オブジェクト指向プログラミング	44
4.1. オブジェクト指向の概要と有用性.....	44
4.2. オブジェクト指向プログラミングの記載方法	46
4.3. オブジェクト指向プログラミングのまとめと練習	54
5. サンプル問題解説	56
5.1. 1～5 問目	56
5.2. 6～10 問目	70
5.3. 11～15 問目.....	81
5.4. 16～21 問目.....	96
5.5. 22～26 問目	115

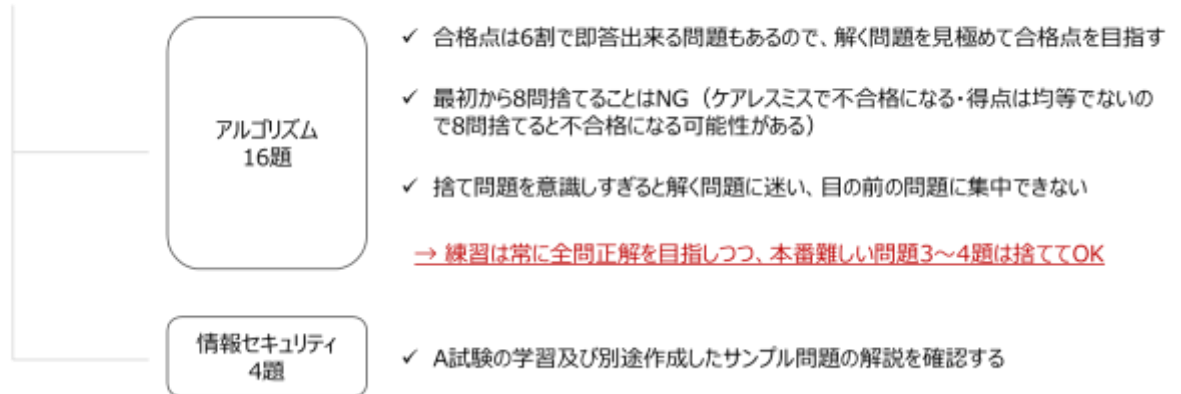
1. アルゴリズム問題の概要と勉強法

1.1. アルゴリズム問題の概要

1.1.1. 科目 B の概要

アルゴリズム問題は基本情報技術者試験の科目 B で出題されます。科目 B は **100 分で 20 問**が出題されますが、そのうちの 16 題がアルゴリズムで、残り 4 題が情報セキュリティです。

B試験（20題・100分）



※情報セキュリティ問題は、A 試験の内容を理解した上、サンプル問題を練習すれば全問正解を目指せるので、別途作成した[科目 B 情報セキュリティサンプル問題の解説動画](#)をご確認ください。

※基本情報技術者試験は 2023 年 4 月に試験形態が変更されましたが、その過去問が非公開である為、2024 年 2 月時点で**公式から公開されているアルゴリズム問題は 26 題**しかありません。試験直前の腕試しとして過去問を取っておきたい人は注意してください。ただし、今は出版社からの予想問題も充実しており、また作成者によって内容が大きく変わるものではないので、公式の過去問にこだわる必要は特にありません。

1.1.2. アルゴリズム問題の概要

アルゴリズム問題では、以下のようなプログラムが出題されます。

〔プログラム〕

```
○整数型の配列: findPrimeNumbers(整数型: maxNum)
整数型の配列: pnList ← {} // 要素数0の配列
整数型: i, j
論理型: divideFlag
for (i を 2 から  まで 1 ずつ増やす)
    divideFlag ← true

    /* i の正の平方根の整数部分が2未満のときは、繰返し処理を実行しない */
    for (j を 2 から i の正の平方根の整数部分 まで 1 ずつ増やす) // α
        if (  )
            divideFlag ← false
            α の行から始まる繰返し処理を終了する
        endif
    endfor
    if (divideFlag が true と等しい)
        pnListの末尾 に i の値 を追加する
    endif
endfor
return pnList
```

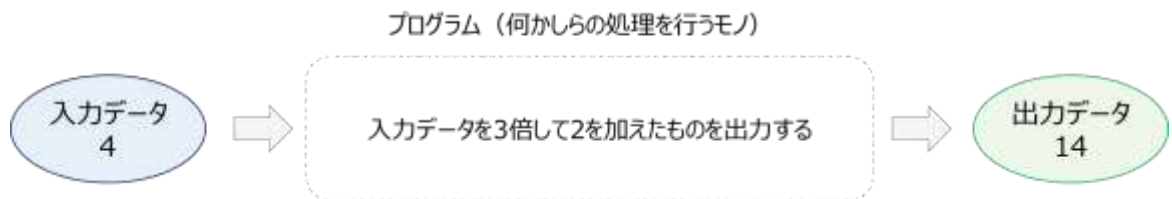
引用：基本情報技術者試験 B試験公開問題1

プログラムはコンピューターに特定の処理を実行させるための命令文書です。コンピューターは人間の言葉を理解できないので、Python や Java といったプログラミング言語を用いて命令を出し、コンピューターはその命令を読み解いて処理を実行します。**プログラムは何かしら便利な処理をしてくれる装置程度**に思ってください。試験では疑似言語と呼ばれる、試験オリジナルのプログラミング言語で命令が記述されます。アルゴリズムはプログラムで記載された命令を実行する為の計算や処理の手順そのものを意味します。

プログラムでは何かしらのデータを入力すると、プログラムで処理が行われ、その結果が出力されます。入れるデータは**引数**、出てくるデータは**戻り値**として定義されることが多いです。また入力データや出力データが無いプログラムも存在します。厳密な理解は不要なので、「**何かを入れたら、処理が行われ、何かが出てくる**」という3点を意識し、たまに例外がある程度に思っておきましょう。



「入力データを 3 倍し、2 を加えたものを出力する」という簡単な処理も立派なプログラムです。
このプログラムに 4 というデータを入力すると、プログラムの処理が行われ、14 が出力されます。シンプルに見えますが、この流れがプログラムの本質です。プログラムが複雑になるほど、細かい処理に目が行ってしまい混乱するケースがありますが、プログラムの原則は「**データを入れたら、便利な処理が行われて何かしらのデータが出てくる**」というシンプルなものです。そのため、この 3 点を理解することでアルゴリズムの全体像を俯瞰して把握し、問題が解きやすくなります。処理を俯瞰する考え方は常に頭に入れておきましょう。



1.1.3. アルゴリズムの出題形式

アルゴリズム問題は例外がありますが、基本は以下のどちらかのタイプが出題されます。

処理結果を問われる：10題

関数 add は、引数で指定された正の整数 value を大域の整数型の配列 hashArray に格納する。格納できた場合は true を返し、格納できなかった場合は false を返す。ここで、整数 value を hashArray のどの要素に格納すべきかを、関数 calcHash1 及び calcHash2 を利用して決める。

手続 test は、関数 add を呼び出して、hashArray に正の整数を格納する。手続 test の処理が終了した直後の hashArray の内容は、 である。

【プログラム】

大域：整数型の配列：hashArray

```
○論理型: add(整数型: value)
整数型: i ← calcHash1(value)
if (hashArray[i] = -1)
  hashArray[i] ← value
  return true
else
  i ← calcHash2(value)
  if (hashArray[i] = -1)
    hashArray[i] ← value
    return true
```

コードの穴埋め：13題

【プログラム】

```
○整数型の配列: findPrimeNumbers(整数型: maxNum)
整数型の配列: pnList ← {} // 素数9の配列
整数型: i, j
論理型: divideFlag
for (i を 2 から  まで 1 ずつ増やす)
  divideFlag ← true

/* i の正の平方根の整数部分が2未満のときは、繰返し処理を実行しない */
for (j を 2 から i の正の平方根の整数部分 まで 1 ずつ増やす) // α
  if (  )
    divideFlag ← false
    α の行から始まる繰返し処理を終了する
  endif
endfor
if (divideFlag が true と等しい)
  pnListの末尾 に i の値 を追加する
endif
endfor
return pnList
```

*問題数は公式から公開されている問題をカウントして算出

それぞれの回答方法は以下の通りです。

処理結果を問われる：10題

<メイン手法>

- ・疑似言語をトレースして回答

<補足的手法>

- ・処理全体像を把握
- ・アルゴリズムを階層的に理解
- ・挙動を一般化

コードの穴埋め：13題

<メイン手法>

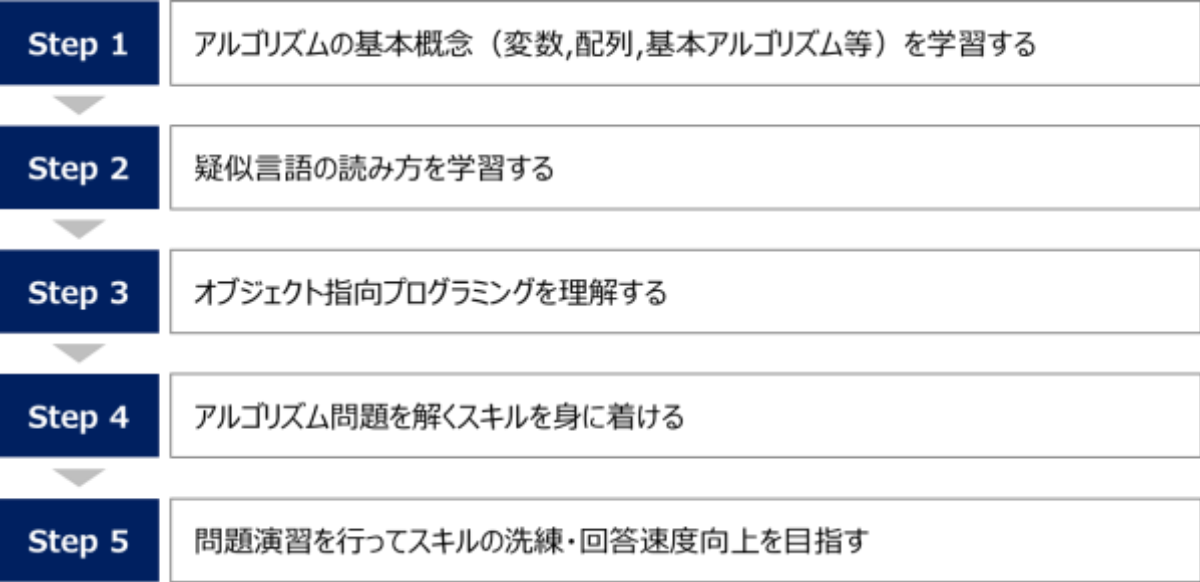
- ・処理全体像を把握
- ・アルゴリズムを階層的に理解
- ・問題文と疑似言語を紐づける
- ・トレースして挙動をイメージする

こちらも、具体的な解き方はサンプル問題の解説で一緒に身に付けていきましょう。

1.2. アルゴリズム問題の対策

1.2.1. アルゴリズム対策の流れ

アルゴリズムは以下の 5 ステップで対策していきます。



それぞれの詳細な説明を以下の表にまとめます。

Step	説明	勉強方法
1	データ構造や頻出アルゴリズムを学習し、プログラムの基本概念を抑える	YouTube 動画
2	疑似言語の文法を学び、プログラムが読めるようになる	
3	オブジェクト指向プログラミングの本質と問題の解き方を学ぶ	
4	26 題の問題演習を通じてアルゴリズム問題を解くスキルを身に着ける	
5	問題練習により、身に着けたスキルを洗練させる	市販の書籍

YouTube 動画では Step1～4 をカバーしており、Step5 は市販の教材で対応していただきます。市販の教材は、2024 年 2 月時点で、[「令和 05 年【下期】基本情報技術者 パーフェクトレーニング過去問題集」](#)がお勧めです。これは試験の過去問集ですが、過去問が公開されていない科目 B については、オリジナル問題を作成してくれています。

なお、2024 年 2 月時点での最新版は令和 06 年度版ですが、これにはオリジナル問題が 2 セットしか含まれておらず、残りは公式からの公開問題です。一方令和 05 年下期版ではオリジナル問題が 3 セット格納されているので、令和 5 年下期版をお勧めしています。（公式からの問題公開状況やパーフェクトレーニングの内容が更新される可能性があるため、必ず最新情報を確認の上書籍を購入してください。）

1.2.2. アルゴリズム勉強のコツ

アルゴリズム勉強の原則として、アルゴリズムは疑似言語を読み解く根本的なスキルを鍛えることが必要です。その為、小手先テクニックや短期間で実力が急上昇する裏技はありません。その一方で、成長は見えにくいものの、勉強を続けることが確実にスキルが身に付きます。また、科目 B アルゴリズムは、[A 試験基礎理論コース](#)の学習さえ完了していれば対策可能なので、基礎理論コース学習後は、**A 試験と科目 B アルゴリズムは並行して学習しましょう**。試験では科目 B で合格点に届かず不合格となるケースが圧倒的に多いので、A 試験と並行して学習を行い、アルゴリズム対策にしっかり時間を割くことが大切です。

具体的な勉強のコツは次の 4 つです。

① 解けなくても限界まで考える



② 最初は時間を気にせず問題を解く



③ まとまった時間が取れる場合はB試験対策



④ 量よりも質を重視した勉強



2. 基本アルゴリズム

2.1. 基本アルゴリズムとは

以下の 3 つのカテゴリに分けて、テストで登場する基本的なアルゴリズムの概念を説明します。

データの持ち方	データ探索方法	データ整列方法
- 変数/配列 - リスト - キュー/スタック - 木構造 (完全2分木/2分探索木/ヒープ木) 	- 線形探索法 2分探索法 - ハッシュ法 	- 基本交換法 - 基本選択法 - 基本挿入法 - シェルソート - クイックソート - ヒープソート - マージソート 
データをどのように保持するか	データをどのように探索するか	データをどのように整列させるか

データの持ち方は問題に直結するので必ず理解してください。一方、データ探索やデータ整列アルゴリズムは「知っていたら役に立つかもしれない」程度なので、丸暗記ではなく知識としてざっくり理解するくらいにとどめておきましょう。アルゴリズムの本番は次の章からなので、この章はアクセル抑えめでいきましょう。

<参考：同じ処理なのに、なぜ色々なアルゴリズムが存在するの？>

データを整列させる為のアルゴリズムとして、基本交換法や基本選択法など、色々な手法を紹介します。「データを整列する」という 1 つの目的しかないのに、なぜ複数のアルゴリズムが存在するのでしょうか。それは、各アルゴリズムの特徴が異なるので、データ量や PC スペックに応じて適切なアルゴリズムが存在するからです。

例えば「東京から九州に旅行する」という目的を達成する際、お金を重視するなら電車で行くし、時間を重視するなら飛行機で行くように、状況に応じて手段を使い分けますよね。それと同様、同じ目的でも、状況に応じてアルゴリズムを使い分けます。ただし、テストではその違いが問われることはないので、細かいことは気にしないで OK です。

2.2. データの持ち方（データ構造）

2.2.1. 変数

変数はデータを一時的に保存しておく名称を持った入れモノです。**変数を用いることで、プログラムを一般化することができます。**例えば、【10+20】という計算をするプログラムを作った場合、このプログラムは10+20しか計算できません。一方、A,Bという入れモノを用意し、【入れモノ A+入れモノ B】というプログラムを作った場合を考えてみましょう。

すると、入れモノ A,B に 10,20 というデータを入れておけば、10+20 という計算が出来ますし、中身を50,60 に変更すれば、50+60 の計算が出来ます。このように、変数を使うことでプログラムの処理を一般化させて、複雑な処理を実現させることが出来ます。

<変数のイメージ>

- $10 + 20 \Rightarrow$ この計算以外出来ない
- 入れモノA + 入れモノB $\Rightarrow$ 入れモノの中身を変更することで任意の数値での計算が可能



変数を使う際は、変数に**データ型**を指定します。変数には、指定されたデータ型以外の値を格納することが出来ません。よく使用されるデータ型は以下の通りです。

データ型	説明
整数型	1,2,3...などの整数を格納
論理型	論理値 true / false のいずれかを格納
文字列型	文字列を格納
文字型	1 文字を格納
クラス型	インスタンスのメモリアドレスを格納（オブジェクト指向で詳しく説明）

論理値について補足します。論理値とは、ある条件式の真偽を表現する値のことです。ある条件式に対し、条件が正しい場合は true、誤っている場合は false が使用されます。プログラミングでは、「点数が30点未満なら生徒に再試験のメールを送り、30点以上なら成績をつける」といった具合に、条件に応じて処理を分岐させることが多いです。そのような条件分岐を作る際に、論理値 true/false を使用します。

条件式を作成する際に使用される演算子は 6 つあります。また、論理値同士の演算を行う演算子として、and、or、not の 3 つがあります。

✓ 条件式を作成する際に使用する演算子

演算子	説明
$A \geq B$	AがBより大きい (A=Bを含む) ときにtrueとなる
$A > B$	AがBより大きい (A=Bを含まない) ときにtrueとなる
$A \leq B$	AがBより小さい (A=Bを含む) ときにtrueとなる
$A < B$	AがBより小さい (A=Bを含まない) ときにtrueとなる
$A = B$	AとBが等しいときにtrueとなる
$A \neq B$	AとBが等しくないときにtrueとなる

✓ 論理値同士の演算で使用する演算子

演算子	説明
and	全ての論理値がtrueのときにtrueとなる
or	1つでもtrueの論理値がある場合にtrueとなる
not	ある論理値と逆の論理値になる

具体例で確認していきましょう。“←”は変数への代入処理で、変数 x という箱に、値を格納するという処理です。また、x〜z は論理型の変数とってください。この時、それぞれの変数の論理値を確認します。

- ① $x \leftarrow \text{and} (4 > 7, 3 \leq 3, 7 \neq 6)$
- ② $y \leftarrow \text{or} (4 > 7, 3 \leq 3, 7 \neq 6)$
- ③ $z \leftarrow \text{not} (x)$

◆x について

演算子 and で 3 つの条件が定義されています。それぞれの条件の論理値は以下の通りです。

- $4 > 7$: false (4 が 7 より大きいという条件は誤り)
- $3 \leq 3$: true (不等号に“=”がついているので正しい)
- $7 \neq 6$: true (7 と 6 が等しくないという条件は正しい)

演算子 and はすべての条件が true の場合に true を返しますが、今回は 1 つ false があります。よって、この条件全体は false となり、変数 x に論理値 false が格納されます。

◆y について

演算子 or は 1 つでも true があれば true を返します。よって、条件全体は true となり、変数 y には論理値 true が格納されます。

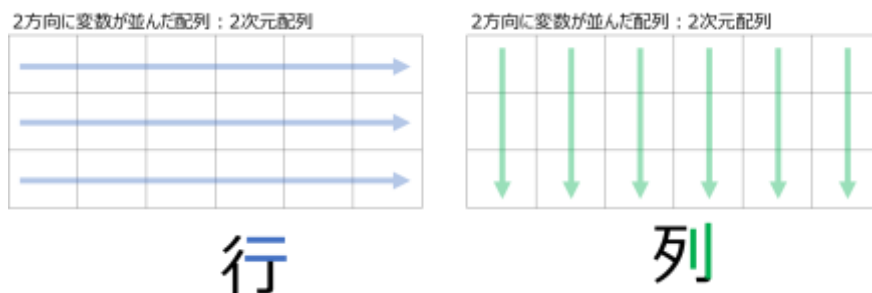
◆z について

演算子 not は対象と逆の論理値を返します。x には false が格納されているので、not (x)は true となり、これが変数 z に格納されます。

2.2.2. 配列

◆配列の概要

配列は**変数を複数個連ねたもの**です。たくさんのデータを格納しておきたいとき配列が使用されます。変数が1方向に並んだ配列のことを1次元配列、変数が縦横の2方向に並んだものを2次元配列と呼びます。2次元配列における**横方向は行**、**縦方向は列**と呼びます。以下のように漢字から連想して覚えると混乱しません。



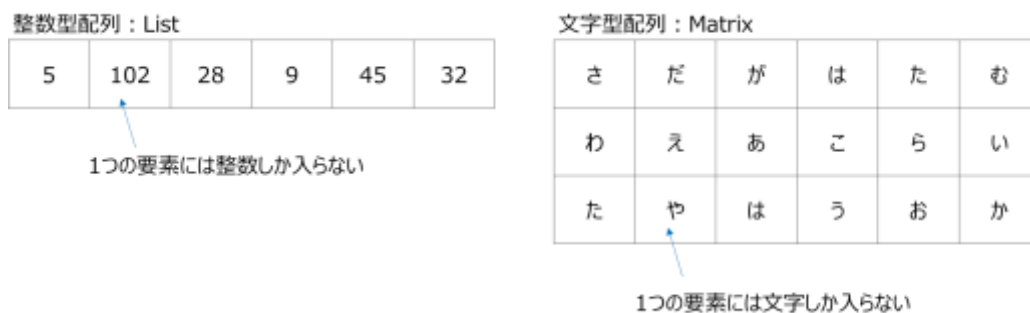
◆配列の要素と表現

配列の中にある1つの入れモノのことを「配列の要素」と呼びます。1次元配列は要素の数を用いて表現されることが多い一方、2次元配列は行数と列数を用いて表現されることが多いです。



◆配列のデータ型

配列は変数と同様に名前とデータ型を保持します。配列の各要素には指定された型のデータしか格納されません。例えば、整数型の1次元配列 List には、各要素に整数しか入りません。文字型の2次元配列 Matrix には、各要素に文字しか入りません。



◆配列の要素番号と要素の指定

配列には各方向に要素番号と呼ばれる番号が割り振られています。要素番号は 0 からスタートすること
もあれば、1 からスタートすることもあり、問題文で必ず指定されます。配列の中の 1 つの要素を指定する
場合、**配列名[要素番号]** で指定します。例えば、28 が格納されている配列要素の番号が 2 な
ので、List[2] = 28 となります。

整数型配列 : List

0	1	2	3	4	5
5	102	28	9	45	32

List[2]

注意点として、整数型配列で**配列内のデータと要素番号が似ている場合**は注意が必要です
例えば、List[3]=1 において、3 が要素番号、1 が要素の中身を示しています。プログラムが複雑にな
ると、要素番号と要素の中身のどちらを指しているのか混乱しやすいので、注意しておきましょう。

整数型配列 : List

0	1	2	3	4	5
6	3	9	1	7	2

List[3] = 1

要素番号とデータの中身が似ている場合、
数字が要素番号とデータのどちらを指しているのか混乱しやすい

二次元配列も同様に、縦方向と横方向に要素番号が振られています。2 次元配列内の要素を指定す
る方法は、**配列名[行の要素番号、列の要素番号]** です。「行・列」の順で要素番号を指定すると覚
えてください。例えば、Matrix[1,2]は 1 行 2 列目の要素 “あ” のことです。

文字型配列 : Matrix

	0	1	2	3	4	5
0	さ	だ	が	は	た	む
1	わ	え	あ	こ	ら	い
2	た	や	は	う	お	か

Matrix[1,2] = あ

2.2.3. リスト

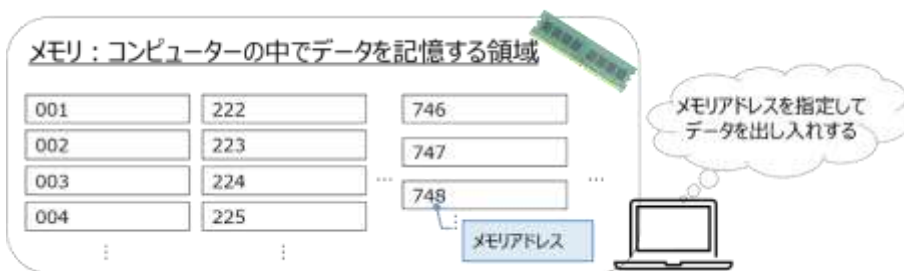
リストとはデータを数珠繋ぎで管理する仕組みで、データとポインタで構成されています。データには、データそのものが保管されており、ポインタに次のリストが保管されているメモリアドレスが記載されています。

リスト（メモリ上に格納）



💡 <補足：メモリアドレスとは？>

メモリとはコンピュータの中でデータを記憶しておく場所のことです。メモリは複数の領域に分割されており、各領域にデータが保存されます。各領域には**メモリアドレス**と呼ばれる番号が割り振られており、コンピュータはメモリアドレスを指定して、データを出し入れします。

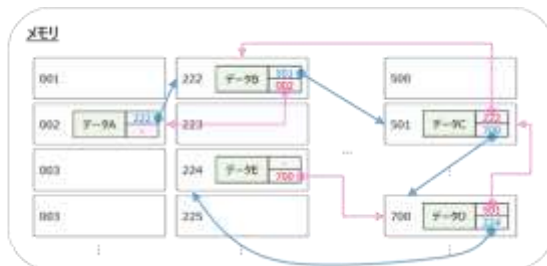


つまり、リストは【**データそのものと、次のデータが入っている場所の情報**】をセットで管理するものです。これを用いて、以下のようにデータを数珠つなぎで管理します。1 方向にしかデータを繋げないリストを**単方向リスト**と呼びます。一方、ポインタを2つもち、前後両方のリストアドレスを保持するリストは前方向と後ろ方向、どちらにもデータを繋ぐので、**双方向リスト**と呼ばれます。

<単方向リスト>



<双方向リスト>



2.2.4. キューとスタック

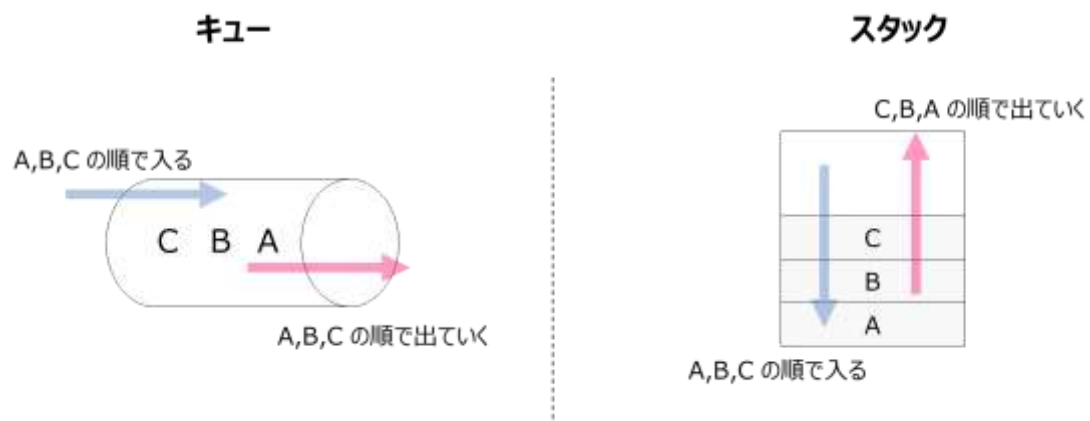
キューとスタックとはデータの出し入れを管理するモノです。キューは先入先出でデータ処理を行うデータ構造、スタックとは後入先出で処理を行うデータ構造のことです。具体的に見ていきましょう。

◆キューのデータ処理

キューに、A,B,C の順でデータが格納されたとします。ここからデータを取り出して処理を実行する際は、**データが格納された順に処理**されます。つまり、データ A,B,C の順で処理されます。このように、先に入ったデータから処理を行うので、先入先出、もしくは英語で表記した First In First Out の頭文字をとって、**FIFO** と呼びます。

◆スタックのデータ処理

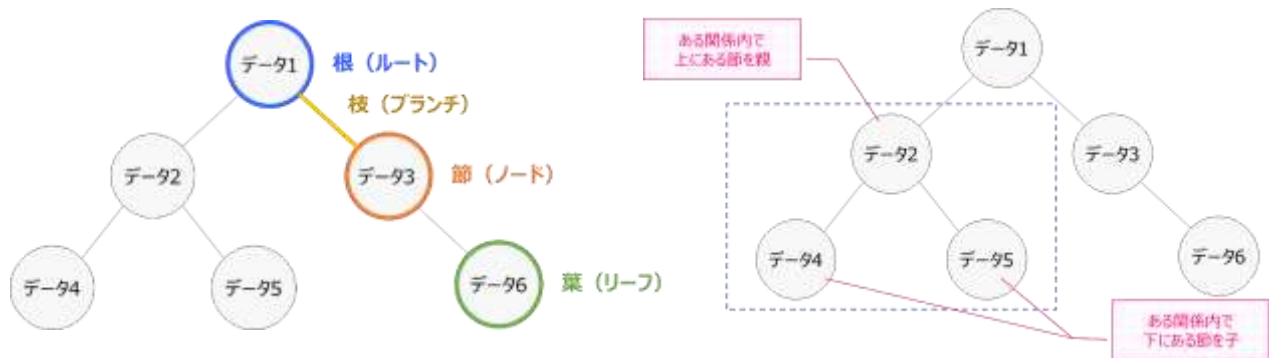
一方、スタックに A,B,C の順でデータが格納された場合は、**データが最後に入った順に処理**を行います。つまり、データ C,B,A の順で処理されます。このように、後に入ったデータから処理を行うので、後入先出、もしくは英語で表記した Last In First Out の頭文字をとって、**LIFO** と呼びます。



この2つはイメージで覚えてしまいましょう。キューは細長い筒で入ったデータから出ていく、スタックは縦に長い入れ物の中でデータが積み重なり、最後に入ったものから出ていくイメージです。キューは横線があるから横に長い、スタック (stack) は日本語に訳すと堆積とか積み重なって意味になるので、縦に長い入れ物でデータが積み重なると覚えてください。

2.2.5. 木構造

木構造はデータを階層構造で保持する方法です。各データの名称は覚えておきましょう。また、ある関係内で上位に存在するデータを親、下位に存在するデータを子と呼びます。



根 or ルート：木構造の一番根元（上）にいるデータ
節 or ノード：根の下にあるデータ
葉 or リーフ：一番最後の節（自分より下にデータが無い節）

木構造は、そのデータの持ち方によって以下のようにさらに細かく定義されています。

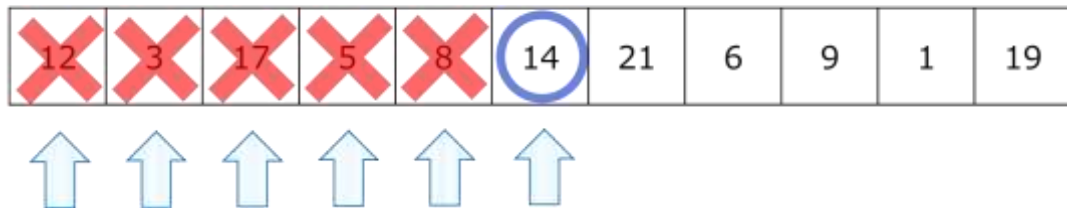


2.3. データ探索アルゴリズム

2.3.1. 線形探索法

ここからは、あるデータ群の中から目的データを探すアルゴリズムを学びます。まずは線形探索法です。これはデータを前から順番にチェックして目的データを探すというシンプルな方法です。例えば、配列の中から14を探したいときは、最初のデータから順にチェックして見つけていきます。

(例) : 下記配列の中から"14"を探す

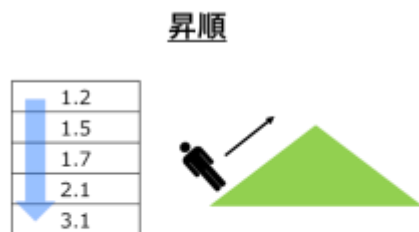


2.3.2. 2分探索法

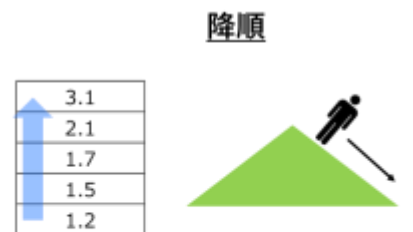
2分探索法は、データが昇順もしくは降順に並んでいる際に使うことが出来る探索方法で、データを2つに分割しながら目的データを探していきます。木構造の1種である2分探索木と名前が似ていますが全く別物なので注意してください。

<💡 補足：昇順と降順とは？>

昇順とは小さい順のことです。山のぼりをイメージすると、小さいふもとから大きい頂上まで昇っていくので小さい順と覚えてください。逆に降順とは大きい順のことです。山の大きなたっぺんから小さいふもとまで下りてくるイメージです。これは知っている前提で問題が出題されるので必ず覚えてください。



小さい順
(山の頂上へ昇っていくイメージ)



大きい順
(山の頂上から降りていくイメージ)

2 分探索法では真ん中の値でデータを 2 分割しながら目的データを探していきます。

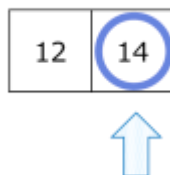
(例) : 下記配列の中から"14"を探す



(例) : 下記配列の中から"14"を探す

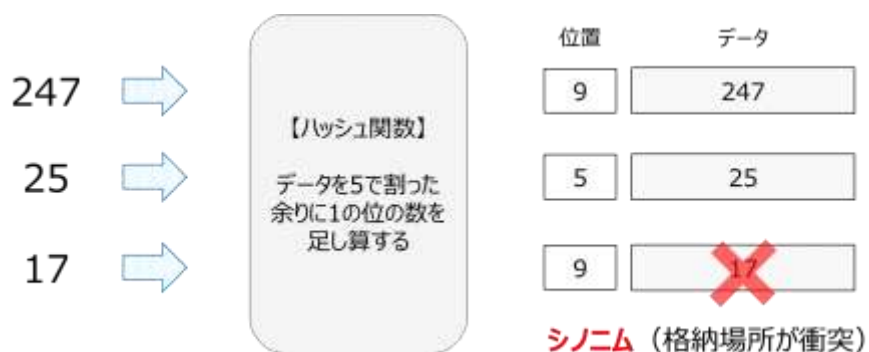


(例) : 下記配列の中から"14"を探す



2.3.3. ハッシュ探索法

ハッシュ探索法では決められた計算式によってデータを計算し、その計算結果の場所を探索する方法です。計算式のことをハッシュ関数と呼びます。例えばハッシュ関数が「データを 5 で割った余りに 1 の位の数を足し算する」とします。これに対し、「247」というデータをどこかに格納する場合、ハッシュ関数の計算で求められる位置（ $247 \div 5$ の余り 2 に、247 の 1 の位の 7 を足した 9）にデータを格納します。このときデータの格納場所が競合するケースをシノニムと呼び、シノニムの発生確率が非常に小さくなるようハッシュ関数を定義する必要があります。



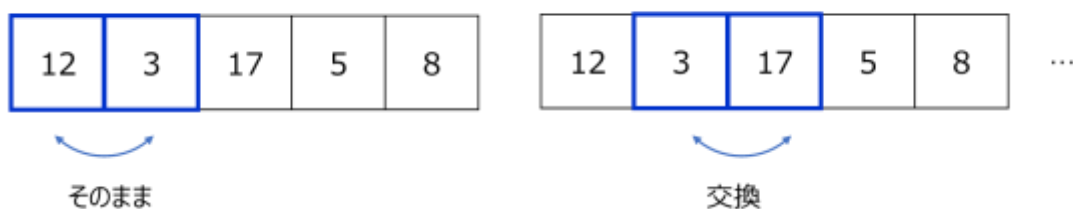
2.4. データ整列アルゴリズム

2.4.1. 基本交換法

データの整列アルゴリズムとは、データを昇順や降順等に並び替えるアルゴリズムです。まずは基本交換法を確認します。基本交換法では隣り合うデータの大きさを比較し、必要に応じて入れ替えていきます。

例えば以下の配列を降順に並べる際、まず 1 番目と 2 番目を比較します。このとき 1 番目の方が値が大きいので入れ替え処理は実施しません。次に 2 番目と 3 番目を見ると、今度は 3 番目の方が数字が大きいので入れ替えます。これを最後まで実施します。

(例) : 下記配列を降順に並び替える



そうすると、このように一番右が一番小さくなります。よって、次は左 4 つで同様の処理を行います。これを最後まで繰り返してデータを整列させます。

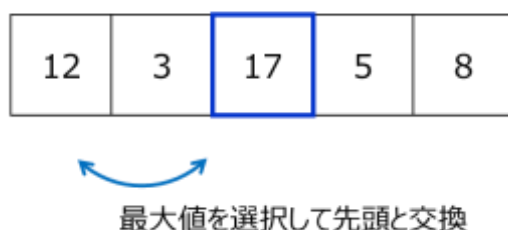
(例) : 下記配列を降順に並び替える



2.4.2. 基本選択法

基本選択は対象データの中から最小値 or 最大値を選択して先頭データと交換することで、データを並び替えていく方法です。

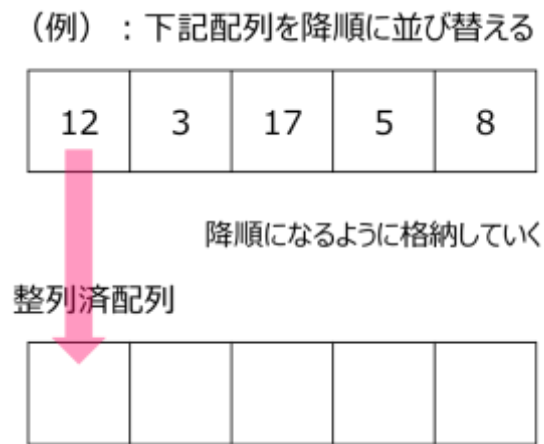
(例) : 下記配列を降順に並び替える



例えば、左の配列では一番大きな 17 を選択して先頭と入れ替えます。すると一番左が一番大きくなるので、残り 4 個でも同様の処理を行っていきます。これを繰り返すことでデータが整列されます。

2.4.3. 基本挿入法

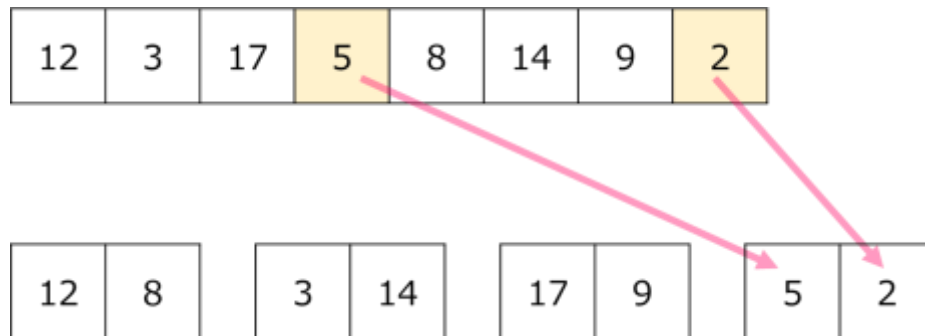
基本挿入法では空の配列を用意して、未整列配列から空の配列の適切な位置へデータを挿入していく方法です。



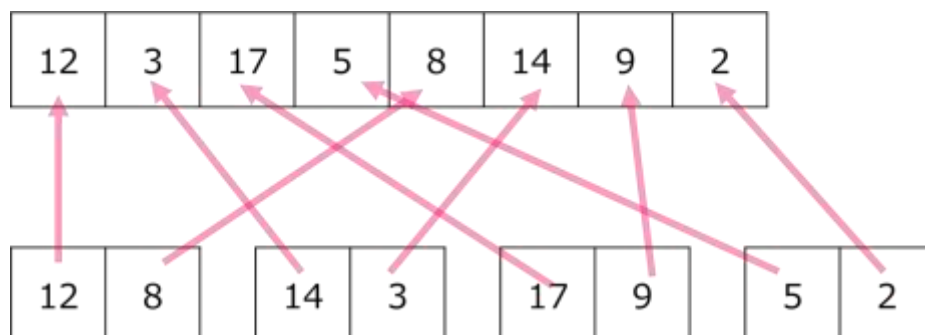
2.4.4. シェルソート

シェルソートはデータを一定間隔ごとに取り出し、取り出したデータを並び替えて元に戻します。そして、取り出し間隔を狭めて再度「データを取り出し並び替えて戻す」という一連の作業を取り出し間隔が 1 になるまで繰り返します。

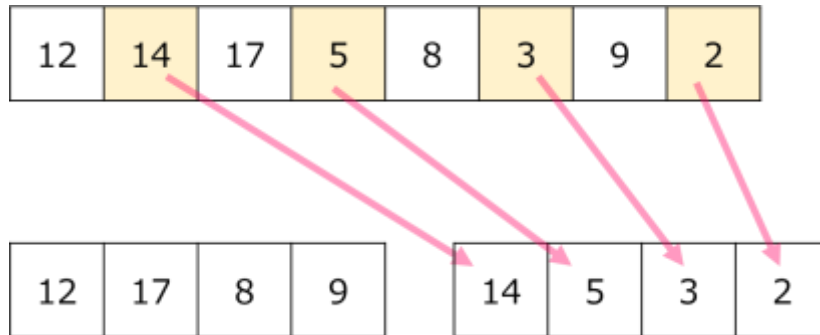
<4 つの間隔でデータを取り出す>



<並び替えて元の配列に戻す>



＜間隔を狭める（2つの間隔）でデータを取り出す＞

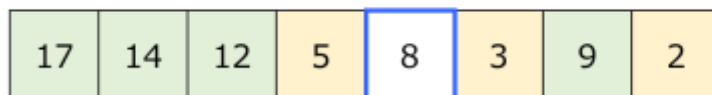


（同様の作業を間隔が1になるまで繰り返す）

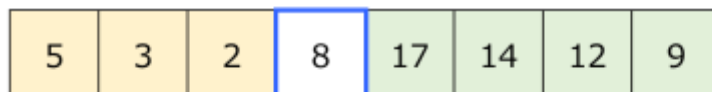
2.4.5. クイックソート

クイックソートはデータ内で基準値を決めて、基準値より大きいデータと小さいデータにグループ分けしながら整列していきます。

（例）：下記配列を昇順に並び替える



基準値（真ん中くらいの値）

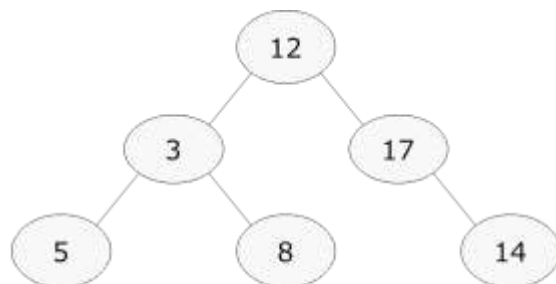
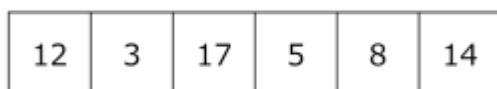


確定

2.4.6. ヒープソート

ヒープソートではデータを木構に直し、木構造がヒープ木となるようにデータを並び替えます。そしてヒープ木の根を配列に格納し直すことでデータの並び替えをします。

（例）：下記配列を降順に並び替える

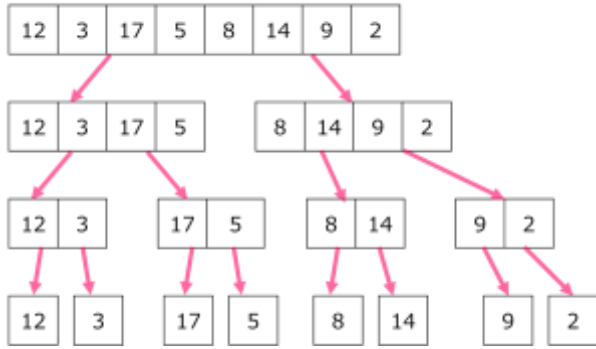


ヒープ木となるように並び替えて配列に戻す

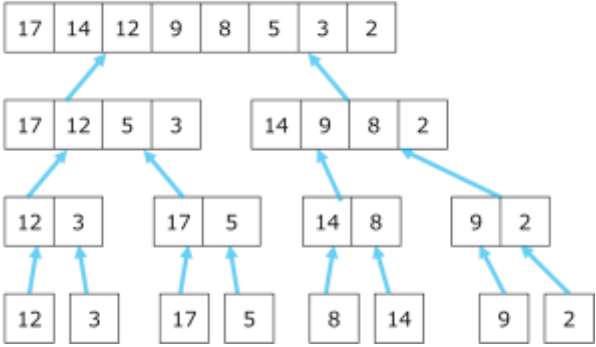
2.4.7. ヒマージソート

マージソートはデータが 1 つになるまで与えられたデータを分割し、分割されたデータを基に戻しなら並び替える方法です。

(例) : 下記配列を降順に並び替える



(例) : 下記配列を降順に並び替える



3. 疑似言語の文法

3.1. 疑似言語の原則

3.1.1. 科目 B の概要

疑似言語とは、プログラムを作成する際に使われる試験オリジナルのプログラミング言語です。初学者が初めて疑似言語を見ると絶望しか感じませんが、1 つ 1 つ読み解くと、意外と理解できます。

```
大域: 整数型の配列: data ← {2, 1, 3, 5, 4}

○sort(整数型: first, 整数型: last)
  整数型: pivot, i, j
  pivot ← data[(first + last) ÷ 2 の商]
  i ← first
  j ← last

  while (true)
    while (data[i] < pivot)
      i ← i + 1
    endwhile
    while (pivot < data[j])
      j ← j - 1
    endwhile

    if (i ≥ j)
      繰返し処理を終了する
    endif
    data[i]とdata[j]の値を入れ替える
    i ← i + 1
    j ← j - 1
  endwhile
  dataの全要素の値を要素番号の順に空白区切りで出力する
  if (first < i - 1)
    sort(first, i - 1)
  endif
  if (j + 1 < last)
    sort(j + 1, last)
  endif
endwhile
```

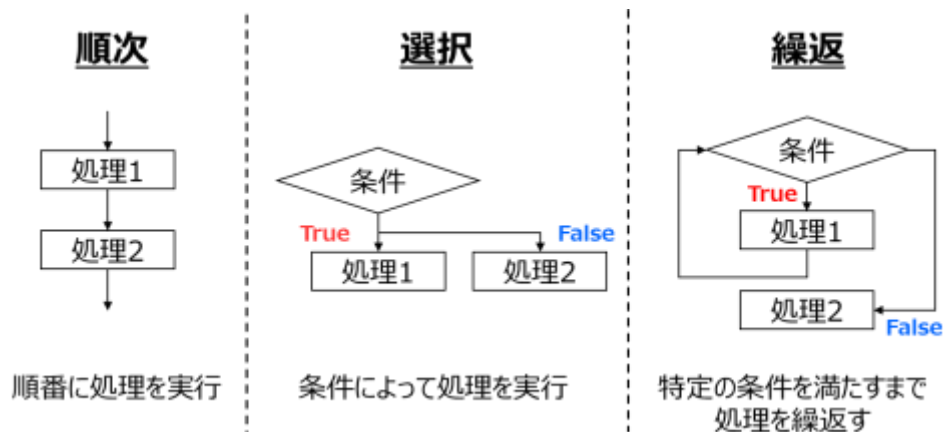
引用: 基本情報技術者試験 科目 B サンプル問題

疑似言語を解釈する際は以下の 2 つの原則があります。

原則 1: 疑似言語の処理は上から順番に実行される

原則 2: 最初に宣言処理を行った後、順次処理・分岐処理・繰返し処理のいずれかを行う

一見複雑な疑似言語も、「順次処理・分岐処理・繰返し処理」の 3 つの処理しか無いと考えると親しみやすくなります。



3.2. 疑似言語の文法

3.2.1. 宣言処理

ここから、具体的に疑似言語の記載方法を確認します。疑似言語はこの文法ルール^{*}に従って記載されるので、文法ルールが理解出来れば疑似言語を読めるようになります。

^{*}文法ルールは問題文に記載されていますが、一々確認していると時間が足りなくなるので、勝手に覚えるまで練習しましょう。

◆プログラムの宣言

文法	○ 手続き名 (引数) ○ 関数名 (引数)
説明	手続きや関数の作成を宣言する処理
補足	・引数はプログラムに入力するデータで、引数無しや複数の引数指定が可能 ・宣言処理を行わないプログラムも存在する ・プログラムの型を指定することで、戻り値のデータ型を指定可能 (詳細は後述)

プログラムの宣言により、コンピューターに「今からこんな名前のプログラムを作るよ」と教えることが出来ます。手続きや関数はプログラムと似たものと思ってください。(厳密には異なりますが、試験ではあまり違いを意識しなくても大丈夫です。) また、“プログラム名()”の中で引数を指定できます。引数はプログラムに入力されるデータです。複数の引数を指定することも可能で、そのときは指定した順に第 1 引数,第 2 引数,...と呼びます。逆に引数が無いプログラムもあります。

< 🔍 実際のコードを確認 >

○append(文字型: qVal)
ListElement: prev, curr
curr ← ListElement(qVal)
if (listHead が a)

引用: 基本情報技術者試験 B試験サンプル問題

(プログラム)
整数型の配列: array ← {1, 2, 3, 4, 5}
整数型: right, left
整数型: tmp
for (left を 1 から (arrayの要素数 ÷ 2 の商) まで 1 ずつ増やす)
right ← 5
tmp ← array[right]
array[right] ← array[left]
5 ← tmp
endfor

引用: 基本情報技術者試験 B試験サンプル問題

“append” という名前のプログラムを宣言しています。プログラム名はヒントになることが多いです。(“append”は日本語で“追加する”という意味なので、データを追加する処理をすると予想できる。) また、引数として文字型変数“qval”を定義しています。

なお、左のコードのようにプログラムが宣言されないケースもあります。宣言をしなくてもプログラム自体を作ることは可能ですが、名前をつけておくと、作ったプログラムを別のプログラムで呼び出して使うことが出来るので便利です。

◆変数/配列の宣言

文法	変数の型：変数名 配列の型 + “の配列”：配列名
説明	変数/配列の型と名称を宣言する処理
補足	・変数や配列を使用する場合は必ず宣言が必要 ・変数は「●●型」、配列は「●●型の配列」と記載される ・プログラムの引数として変数/配列が定義されているケースもある ・宣言時に初期値を代入することも可能

< 🔍 実際のコードを確認 >

【プログラム】

```
整数型の配列: array ← {1, 2, 3, 4, 5}
整数型: right, left
整数型: tmp
```

引用：基本情報技術者試験 B試験サンプル問題

宣言処理により整数型の配列 array と、整数型の変数 right, left, tmp が生成されています。プログラムの宣言で確認したように、プログラムの引数として宣言されるケースもあります。

また、“array ← {1,2,3,4,5}” では配列 array に初期値としてデータを代入する処理です。配列の宣言と同時に、array には 1,2,3,4,5 が入るということです。変数や配列では宣言と同時にデータを入れることが出来ず、逆に空っぽのままでも OK です。

また、疑似言語では配列は{ }で定義されており、2次元配列では1つの{ }が1行を意味します。

【配列】

配列の要素は、“[”と“]”の間にアクセス対象要素の要素番号を指定することでアクセスする。なお、二次元配列の要素番号は、行番号、列番号の順に“,”で区切って指定する。

“(”は配列の内容の始まりを、“)”は配列の内容の終わりを表す。ただし、二次元配列において、内側の“(”と“)”に囲まれた部分は、1行分の内容を表す。

引用：試験で使用する情報技術に関する用語・プログラム言語など

{ 1,1,1 }



1次元配列

1	1	1
---	---	---

{ {1,2,3} , {4,5,6} , {7,8,9} }

2次元配列の1行



2次元配列

1	2	3
4	5	6
7	8	9

◆注釈

文法	/* 注釈 */ or //注釈
説明	注釈の記述
補足	・注釈はプログラムの処理に影響を及ぼさない補足説明であり、問題のヒントになる

3.2.2. 順次処理（代入処理と呼出処理）

基本処理の 1 つである順次処理を確認します。順次処理は代入処理と呼出処理の 2 つです。

◆代入処理

文法	変数名 ← 式 配列名 ← 式
説明	変数や配列にデータを格納する処理

< 🔍 実際のコードを確認 >

```
○整数型: fee(整数型: age)
整数型: ret
if (age が 3 以下)
    ret ← 100
elseif (  )
    ret ← 300
```

引用：基本情報技術者試験 B試験サンプル問題

代入処理は順次処理の 1 つです。変数や配列にデータを格納します。“式”と書いてある部分は、“何かしらのデータ”と読み替えて OK です。左のサンプルコードでは宣言された整数型変数 ret に 300 を代入しています。

◆呼出処理

文法	手続き名（引数） 関数名（引数）
説明	別途定義されたプログラムを呼び出して使用する
補足	プログラムの宣言と似ているが、宣言処理では「○」が必要

< 🔍 実際のコードを確認 >

```
if (i ≥ j)
    繰返し処理を終了する
endif
data[i]とdata[j]の値を入れ替える
i ← i + 1
j ← j - 1
endwhile
dataの全要素の値を要素番号の順に空白区切りで出力する
if (first < i - 1)
    sort(first, i - 1)
endif
```

引用：基本情報技術者試験 B試験サンプル問題

あるプログラムの中で、別途作成したプログラムを使用することが出来ます。左のプログラムの中で、別途定義された“sort”というプログラムが呼び出されています。この処理により、プログラム sort が起動して処理を行います。「first, i-1」という 2 つの引数入力データとなり、何かしらの処理を行って、結果を出力します。

3.2.3. 分岐処理 (if 文)

次に分岐処理を確認します。分岐処理は if 文しかありません。

◆if 文

文法	if (条件式 1) 処理 1 elseif (条件式 2) 処理 2 elseif (条件式 n) 処理 n else 処理 n+1 endif
説明	・条件式に応じて処理を制御する構文で、if から endif までの一塊を指す ・条件式を上から順にチェックし、最初に true となった条件に対応する処理を行う ・処理を行った場合は残りの条件の確認はせず、処理から抜ける ・どの条件にも該当しない場合は else に記載されている処理を行って処理から抜ける
補足	・elseif は複数記載することも、逆に 1 つも記載しないことも可能 ・else の処理は省略可能 (どの条件にも該当しない場合は処理が実施されない)

< 🔍 実際のコードを確認 >

```
整数型 : score
文字型 : grade
if (score が80以上)
    grade ← "優"

elseif (score が70以上80未満)
    grade ← "良"

elseif (score が60以上70未満)
    grade ← "可"

else
    grade ← "不可"
endif
return grade
```

左のコードは変数 score に代入された点数に応じた成績が変数 grade に代入される処理です。score=75 のときを考えましょう。if 文では条件式を上から上から順にチェックするので、以下の流れで処理されます。

- (1) <条件 1> score が 80 以上 : false
→ 次の処理に移る
- (2) <条件 2> score が 70 以上 80 未満 : true
→ 対応する処理に移り、grade に "良" を代入
- (3) 条件にヒットしたので残りの処理は行わずに if 文から抜け出す (endif の次の処理に移る)
- (4) return で "良" を返す

なお、“return” は**戻り値を返す処理**です。戻り値とは「プログラムから出てくるモノ」程度の認識で大丈夫です。return 処理では、プログラムを終了させ、処理結果を返してくれます。

<💡 補足：プログラムの終了>

プログラムが終了するケースは以下の 2 パターンなので知っておきましょう。

- ① return 処理を行う
- ② 記載されている全ての処理が完了する

また、elseif は何個記載しても良いし、逆に記載しなくても良いです。elseif を増やすほど条件分岐を増やすことが出来ます。また、else の処理は省略可能であり、その場合、いずれの条件にも該当しないときは何も処理を実施せずに if 文から抜け出します。つまり、最もシンプルな if 文は 3 行で終わります。

① elseifは何個記載してもよい

```
if (score が90以上)
  grade ← "S"
elseif (score が80以上90未満)
  grade ← "A"
elseif (score が70以上80未満)
  grade ← "B"
elseif (score が60以上70未満)
  grade ← "C"
elseif (score が50以上60未満)
  grade ← "D"
else
  grade ← "不可"
endif
```

条件分岐が増える

② elseifは記載しなくてもよい

```
if (score が60以上)
  grade ← "合格"
else
  grade ← "不合格"
endif
```

条件分岐が減る

③ elseは記載しなくてもよい

```
if (score が90以上)
  grade ← "S"
elseif (score が80以上90未満)
  grade ← "A"
elseif (score が70以上80未満)
  grade ← "B"
elseif (score が60以上70未満)
  grade ← "C"
elseif (score が50以上60未満)
  grade ← "D"
endif
```

条件が全てFalseの場合処理を行わない

最もシンプルな if 文

3.2.4. 繰返処理 (while 文 / do 文 / for 文)

次に繰返処理を確認します。繰返処理は while 文、do 文、for 文の 3 つがあります。

◆while 文

文法	while (条件式) 処理 endwhile
説明	・while 文は while から endwhile までの一塊を指す ・条件が true である限り処理を繰り返し、条件が false になると処理から抜け出す
補足	・無限ループとなる処理を記載してはいけない

< 🔍 実際のコードを確認 >

```
while (low ≤ high)
  middle ← (low + high) ÷ 2 の商
  if (data[middle] < target)
    low ← middle
  elseif (data[middle] > target)
    high ← middle
  else
    return middle
  endif
endwhile
```

引用：基本情報技術者試験 B試験サンプル問題

while～endwhile までの 1 塊が while 文であり、条件 $low \leq high$ を満たす限り中にある処理を繰り返します。

条件が true の時に処理を実行するルールは if 文と同様なので、セットで覚えておきましょう。

注意点として、無限ループになってしまう処理は NG です。例えば以下のコードでは、変数 count に 5 が入っており、「count が 0 より大きい」という条件で while 文の繰返し処理を行っています。

左側のコードでは、while 文の中の処理は count に対して操作を行っておらず、count の値が変わりません。つまり、**count の値は永遠に 5 なので while 文の条件式が常に true となり、処理が無限に続いてしまいます**。これを防ぐため、処理の中に count の値を 1 減らす等、条件式に影響を与える処理が必要です。こプログラムの穴埋め問題で役立つテクニックなので、理解しておきましょう。

NGな繰返し処理の例（無限ループ）

```
count ← 5
while (countが0より大きい)
  if (iがjより大きい)
    price[i] ← totalprice[j]
  endif
endwhile
```

While文の処理でcountの値が変わらない
条件式 (count>0) を満たせず無限ループに入ってしまう

OKな繰返し処理の例

```
count ← 5
while (countが0より大きい)
  if (iがjより大きい)
    price[i] ← totalprice[j]
  endif
  count ← count - 1
endwhile
```


◆while 文

文法	do 処理 while (条件式)
説明	・do 文は <u>do から while までの一塊</u> を指す ・最初に処理を行い、次に条件が true である限りその処理を繰り返す
補足	・無限ループとなる処理を記載してはいけない

do 文は試験にあまり出題されず、また while 文と似ているので簡単に済ませます。do 文では while 文と異なり、**最初に処理を実行してから条件式に移ります**。そして、while の条件が true の場合に処理を繰り返します。最初に処理を実行すること以外は while 文と同様の処理です。

◆for 文

文法	for (制御記述) 処理 endfor
説明	・for 文は <u>for から endfor までの一塊</u> を指す ・制御内容を基に繰り返し実行する
補足	・繰返処理は for 文と while 文どちらでも記載できるが、処理によって相性がある

< 🔍 実際のコードを確認 >

```
○整数型の配列: makeNewArray(整数型の配列: in)
整数型の配列: out ← {} // 要素数0の配列
整数型: i, tail
outの末尾 に in[1]の値 を追加する
for (i を 2 から inの要素数 まで 1 ずつ増やす)
    tail ← out[outの要素数]
    outの末尾 に (tail + in[i]) の結果を追加する
endfor
return out
```

引用: 基本情報技術者試験 B2試験サンプル問題

制御内容は「i を 2 から in の要素数まで 1 ずつ増やす」です。in は整数型配列として宣言されています。つまり、最初に i=2 として、for 文の中の処理を実施し、次に i の値を 1 増やし、i=3 として処理を実施します。これを繰り返し、i の値が配列 in の要素数になるまで処理を行います。

なお繰返処理は for 文/while 文のどちらを使っても記述することが出来ますが、処理の内容によって相性があるので、2 つの文法が存在します。

3.3. 疑似言語の読み方確認

ここまで確認した文法がどのように試験で記載されるか、サンプル問題を使って簡単に確認しましょう。詳細は問題演習の章で解説するので、ここでは読み方を整理していきます。

```
○整数型配列の配列: transformSparseMatrix(整数型の二次元配列: matrix)
  整数型: i, j
  整数型配列の配列: sparseMatrix
  sparseMatrix ← {{}, {}, {}} /* 要素数0の配列を三つ要素にもつ配列 */
  for (i を 1 から matrixの行数 まで 1 ずつ増やす)
    for (j を 1 から matrixの列数 まで 1 ずつ増やす)
      if (matrix[i, j] が 0 でない)
        sparseMatrix[1]の末尾 に iの値 を追加する
        sparseMatrix[2]の末尾 に jの値 を追加する
        sparseMatrix[3]の末尾 に matrix[i, j]の値 を追加する
      endif
    endfor
  endfor
  return sparseMatrix
```

引用: 基本情報技術者試験 B試験サンプル問題 問4

最初にプログラム transformSparseMatrix が宣言されています。ここで、変数や配列のように、プログラムでも型が指定されています。このように**プログラム宣言時に型が指定される場合、これは戻り値（出てくるモノ）の型**を指定しています。プログラムの戻り値は sparseMartix なので、これが整数型の配列であることがプログラム宣言時に示唆されています。

```
○整数型配列の配列: transformSparseMatrix(整数型の二次元配列: matrix)
  整数型: i, j
  整数型配列の配列: sparseMatrix
  sparseMatrix ← {{}, {}, {}} /* 要素数0の配列を三つ要素にもつ配列 */
  for (i を 1 から matrixの行数 まで 1 ずつ増やす)
    for (j を 1 から matrixの列数 まで 1 ずつ増やす)
      if (matrix[i, j] が 0 でない)
        sparseMatrix[1]の末尾 に iの値 を追加する
        sparseMatrix[2]の末尾 に jの値 を追加する
        sparseMatrix[3]の末尾 に matrix[i, j]の値 を追加する
      endif
    endfor
  endfor
  return sparseMatrix
```

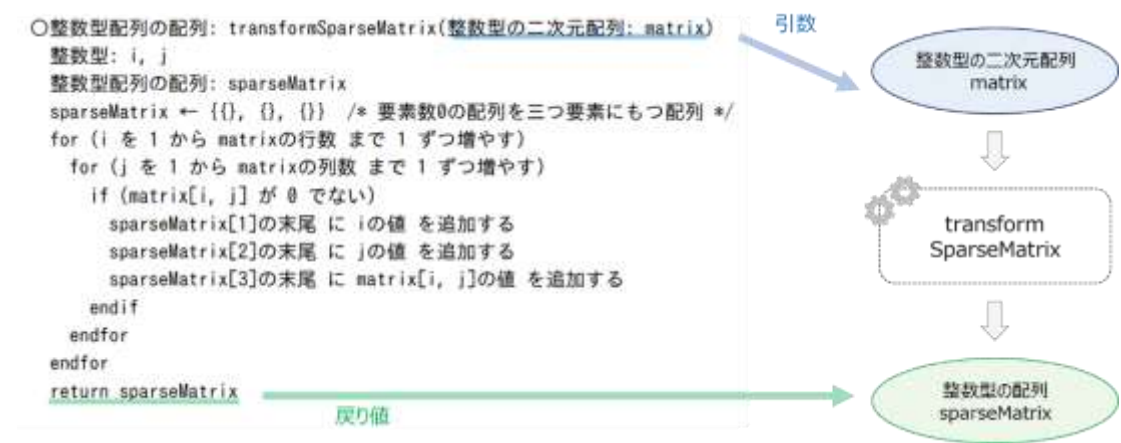
プログラムの宣言時の型指定は戻り値の型を指定

戻り値 sparseMatrixは整数型の配列であることがわかる

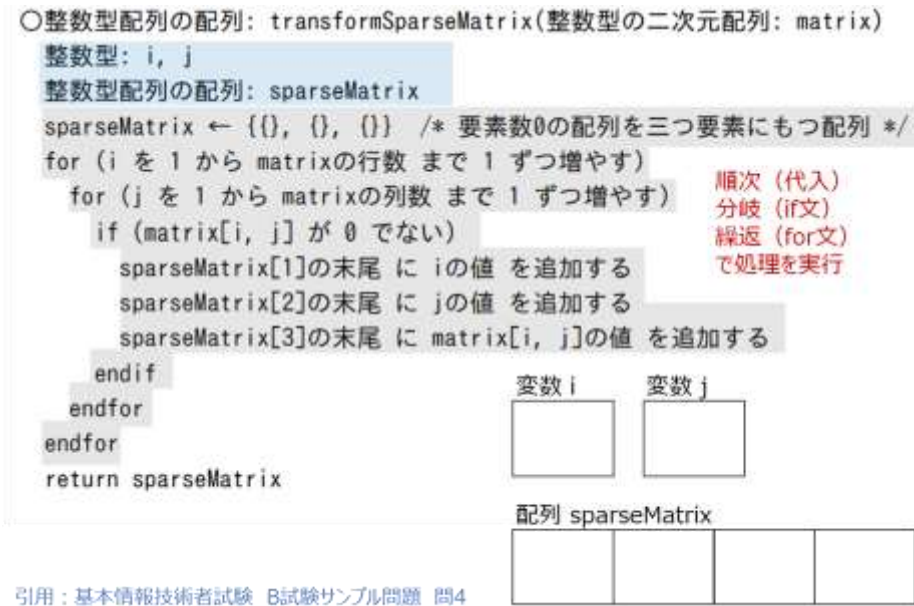
<💡 補足: プログラムの型指定は必要? >

宣言処理の中で sparseMatrix 配列は整数型配列と指定されています。その為、プログラムで宣言しなくても戻り値の型は分かるのですが、お作法としてプログラムの型が宣言されています。

また、プログラムは「何かを入ると何かが出てくる」のです。入れるモノは プログラム名()と宣言されている"()"の中で引数として指定されます。その為、このプログラムでは二次元配列 matrix をプログラムに入れていることがわかります。出てくるモノは return で戻り値として返されている配列 sparseMatrix です。



次に、変数 i, j と配列 sparseMatrix の宣言がされています。変数と配列はデータを格納する入れモノで、プログラムを実行する際に使われます。その下に transformMatrix の処理が記載されています。ここではこれまで見てきたような順次処理、条件分岐処理、繰返処理が組み合わさっています。この処理を上から順に処理していくことで、プログラムが実行されます。



引用：基本情報技術者試験 B試験サンプル問題 問4

3.4. 疑似言語のトレース

3.4.1. トレースの方法

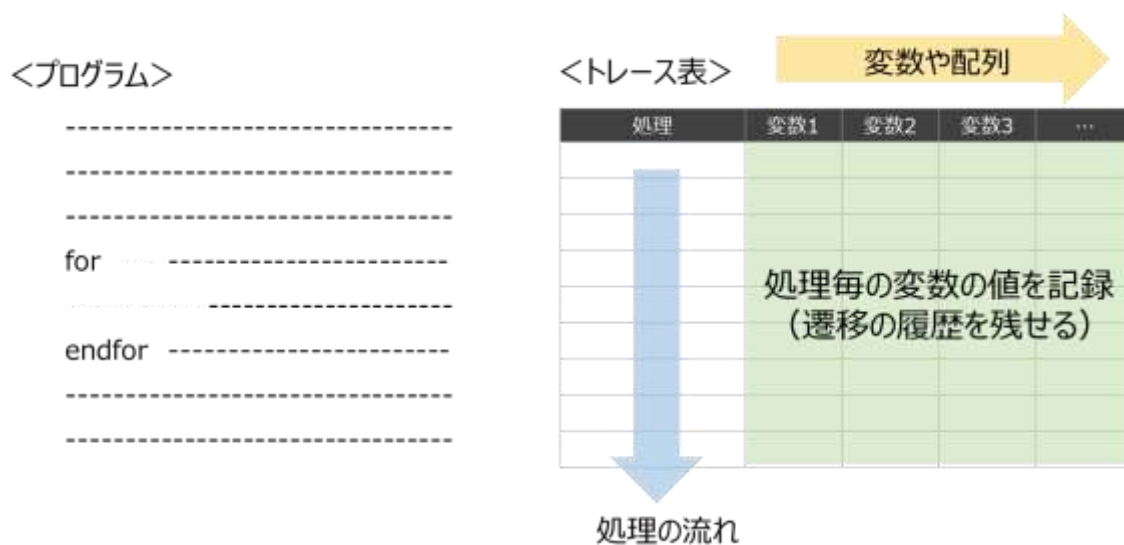
前の節でサンプル問題の記載方法は理解出来ましたが、プログラムがどのように動いていくかはイメージできませんよね。そこで使われるテクニックがトレースです。トレースとは、**プログラムを実行した際の変数の値を 1 行ずつ確認し、プログラムの挙動を確認する手法**です。アルゴリズムをトレースすることで処理の挙動をイメージしたり、また問題によってはトレースだけで回答を導くことも出来るので、アルゴリズム問題において、基本的かつ必須のスキルです。試験でもトレース能力の重要性が強調されています。

トレースをする際は、トレース表を作ります。慣れてきたり、問題によってはトレース表を作らないケースもありますが、慣れるまでは必ずトレース表を作るものと思ってください。行列には以下の内容を記述します。

トレース表の行：プログラムの処理を 1 行ずつ記載

トレース表の列：プログラムで登場する変数や配列を並べる

そして、1 行ごとの変数や配列の値の遷移を表の中に記載します。こうすることで、変数の移り変わりを記録することが出来ます。



サンプルコードを用いてトレースのやり方を見ていきましょう。以下のプログラムを実行したとき、変数 C に入っている値をトレースで確認していきます。ただし、いきなりトレースを行うのではなく、まずはプログラムの構成を階層的に確認します。動画を参考に、コードの階層を以下にメモしてみましょう。

```
整数型 : A ← 5
整数型 : B ← 15
整数型 : C ← 12
整数型 : i

for ( i を1から3まで1ずつ増やす )
    A ← A + 4

    if ( AがB以上 )
        C ← C + 20
    endif

    C ← C mod A
endfor
```

<プログラムの階層構造>

- ・変数宣言で、変数 A〜C 及び i が作成（A〜C には初期値を格納）
- ・for 文で、制御内容に応じて中の処理を繰り返す
- ・for 文の中は、代入処理が 2 つと、if 文の処理の合計 3 つの処理が記載

<💡 補足：演算子 mod>

C mod A とは、C を A で割った際の余りです。例えば、11 mod 4 は、11÷4 のあまり 3 が出力されます。mod は足し算や割り算と同様の演算子として出題されるので、覚えておきましょう。

では以下のようなトレース表に値を埋めていきます。余裕がある方はご自身でトレースしてみてください。

処理	i	A	B	C

トレースした結果は以下の通りです。トレースの過程は動画を確認し、気づいたポイントがあればメモ欄に記載しておきましょう。

問題：以下のプログラムを実行した際のCの値は？

```
整数型 : A ← 5
整数型 : B ← 15
整数型 : C ← 12
整数型 : i
for ( i を1から3まで1ずつ増やす )
    A ← A + 4
    if ( AがB以上 )
        C ← C + 20
    endif
    C ← C mod A
endfor
```

処理	i	A	B	C
初期状態		5	15	12
i =1 の繰り返し処理	1	9	15	3
i =2 の繰り返し処理	2	13	15	3
i =3 の繰り返し処理	3	17	15	6

 ～メモ欄～

また、トレースについて以下の 4 つの補足を確認しておきましょう。

1	全体像把握の意識を忘れない	・ トレースは細かい処理を1つずつ確認するテクニックなので機械的作業になりやすい ・ プログラムの目的を意識してトレースすることで精度が上がり、アルゴリズムの感覚も身に付きやすい
2	繰り返し処理で特に有効	・ 繰り返し処理では変数の変化が激しく、複雑になりやすい為、値遷移の履歴を保持出来るトレースは非常に有効 ・ 順次処理などは敢えてトレース表を作らなくても理解できるケースが多い
3	具体例を使ってトレースする	・ 問題文に具体例が提示されていない場合、自分で具体例を作成してトレースする
4	トレース表を使わないこともある	・ トレース表はあくまで問題を解くためのツールなので、問題によってはトレース表なしで回答することもある（ただし、トレース表を作成できるスキル自体は必須）

3.4.2. トレースの問題演習

理解を深めるため、オリジナルの問題演習を 4 題行うので、それぞれのコードをトレースしてみてください。
練習問題ではプログラムの挙動を問題文に書かないので、トレースをしながらプログラム全体の挙動理解にも挑戦してみてください。また、サンプル問題の配列要素番号は全て 1 スタートとなっていますが、試験で要素番号 0 スタートの問題が出題されても焦らないよう、練習問題では敢えて要素番号を 0 スタートとしています。

（注）問題と解答のみを記載します。詳しい解説は動画を確認し、必要に応じてメモしてください。

◆問 1（問題） ※配列の要素番号は 0 スタートとする

- 以下のプログラムを実行した際、totalに格納されている値は？またこれはどんなプログラム？

整数型配列 : points ← {65, 70, 85, 90, 75}

整数型: total, i

for (i を0から 配列pointsの要素数-1 まで1ずつ増やす)

total ← total + points[i]

endfor

< 📄 回答用紙 >

◆問 1 (回答)

整数型配列 : points

0	1	2	3	4
65	70	85	90	75

処理	i	total	points[i]
初期状態			
i = 0	0	65	65
i = 1	1	135	70
i = 2	2	220	85
i = 3	3	310	90
i = 4	4	385	75

<答え>

total : 385

プログラム : 配列pointsの要素を前から1つずつ合計する処理

◆問 2（問題） ※配列の要素番号は 0 スタートとする

- プログラムを実行した際の変数 total の値は？またこれはどんなプログラム？
※プログラムで使われている add の説明をふまえて回答すること

<プログラム add の説明>

引数：整数型の変数 x,y

処理：xとyを加算した結果を返す

整数型配列：numbers ← {5, 3, 10, 1, 4}

整数型: total ← 0

整数型：index ← 0

while (index が numbersの要素数より小さいとき)

total ← add(total, numbers[index])

index ← index + 1

endwhile

<  回答用紙 >

◆問 2 (回答)

整数型配列 : numbers

0	1	2	3	4
5	3	10	1	4

処理	index	total	numbers[index]
初期状態	0	0	5
index = 0	1	5	3
index = 1	2	8	10
index = 2	3	18	1
index = 3	4	19	4
index = 4	5	23	-

<答え>

total : 23

プログラム : 配列numbersの要素を前から1つずつ合計する処理

◆問 3 (問題) ※配列の要素番号は 0 スタートとする

■ プログラムを実行した際、配列arrの中身は？ またこれはどんなプログラム？

整数型配列 : arr ← {14, 5, 9, 28}
整数型: tmp, i, j

```
for (i を 0 から配列 arr の要素数 - 2 まで増やす)
  for (j を 0 から配列 arr の要素数 - i - 2 まで増やす)
    if (arr[j] < arr[j + 1])
      tmp ← arr[j]
      arr[j] ← arr[j + 1]
      arr[j + 1] ← tmp
    endif
  endfor
endfor
```

<  回答用紙 >

◆問 3（回答）

整数型配列：arr

0	1	2	3
28	14	9	5

整数型 tmp

9	
---	--

処理	i	j	tmp	arr[j]	arr[j+1]
i = 0, j = 0	0	0		14	5
i = 0, j = 1	0	1	5	9	5
i = 0, j = 2	0	2	5	28	5
i = 1, j = 0	1	0	5	14	9
i = 1, j = 1	1	1	9	28	9
i = 2, j = 0	2	0	14	28	14

<答え>

arr : {28,14,9,5}

プログラム：配列arrの要素をバブルソートを使用して降順に並び替える

◆問 4 (問題) ※配列の要素番号は 0 スタートとする

- プログラムを実行した際の変数 count の値は？またこれはどんなプログラム？
※プログラムで使われている count_if の説明をふまえて回答すること

＜プログラム count_if の説明＞

引数: 整数型の値 x

処理: xが5より大きい場合に1を返し、そうでなければ0を返す

整数型配列: matrix ← {{4, 7}, {10, 2}, {21, 6}}

変数名: count, i, j

```
for (i を 0 から配列 matrix の行数 - 1まで1ずつ増やす)
  for (j を 0 から配列 matrix の列数 - 1まで1ずつ増やす)
    count ← count + count_if(matrix[i, j])
  endfor
endfor
```

＜📄 回答用紙＞

◆問 4 (回答)

整数型配列 : matrix

	0	1	処理	i	j	count	matrix[i,j]
0	4	7	i = 0, j = 0	0	0	0	4
			i = 0, j = 1	0	1	1	7
1	10	2	i = 1, j = 0	1	0	2	10
			i = 1, j = 1	1	1	2	2
2	21	6	i = 2, j = 0	2	0	3	21
			i = 2, j = 1	2	1	4	6

<答え>

count : 4

プログラム : 配列matrixの中で5より大きい要素の数をカウントする

4. オブジェクト指向プログラミング

4.1. オブジェクト指向の概要と有用性

ここからオブジェクト指向プログラミングに入ります。オブジェクト指向プログラミングとは、クラスからインスタンスを作成して、プログラミングを効率化する手法です。

RPG ゲームのキャラクターを作るケースを基に、オブジェクト指向プログラミングの有用性を確認しましょう。私たちは RPG ゲームを開発するエンジニアで、アプリ内のキャラクターを設計するとします。このとき、以下のような対応を行います。

	名称：さる	データ	<データの作成> ・名前を格納する変数を用意・名前格納 ・HP を格納する変数を用意・HP 格納 ・（...以下同様に、必要データ作成）
	体力：30		
	魔力：20		
	...		
	Xボタンでパンチ	アクション	
Yボタンで防御			
Zボタンでひっかく			
...			

これでようやく 1 つのキャラクターが作成出来ます。2 つ目のキャラクターも同様に、0 からデータとアクションのプログラムを作成し、これを全てのキャラクターで繰り返します。このように全て 0 から作ることは非効率で大変であることがわかりますね。

キャラクターの 共通事項をまとめた設計書



データ	名称：	アクション	Xボタンでパンチ
	体力：		Yボタンで防御
	魔力：		

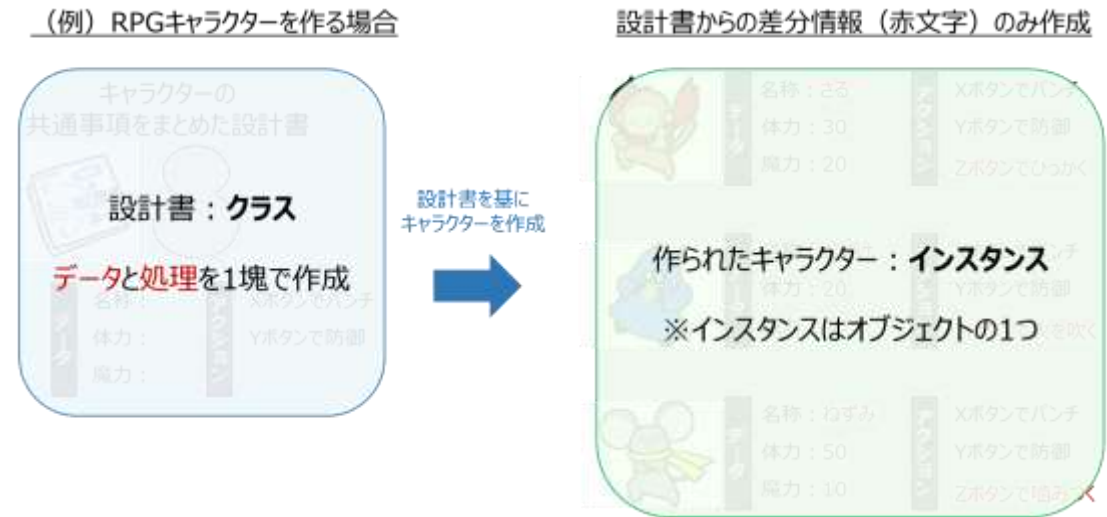
そこで、オブジェクト指向プログラミングでは、最初に**全てのキャラクターの共通事項をまとめた設計書**を作ります。

具体的には、各キャラクターは名前や HP などのデータを保持しており、また、X ボタンでパンチ、Y ボタンで防御という共通のアクションを保持しているので、その共通事項を設計書に落とし込みます。そして、各設計書を元に具体的なキャラクターを作成します。

開発する際は、まずは設計書からキャラクターの基を作成します。そこに各キャラクターの名前や具体的なステータス情報を追加したり、キャラクター固有の特殊行動を追加するなど、設計書からの差分だけ実装します。こうすることで、効率的にキャラクターを開発することが出来ます。



この時の設計書を**クラス**と呼びます。クラスは名前や HP などの「**データ**」と攻撃や防御といった「**処理**」を1カタマリにして作成されます（ココは大切！）。また、設計書から作られた具体的なキャラを**インスタンス**と呼びます。ちなみに、オブジェクト指向プログラミングでは、「開発されたすべてのもの」をオブジェクトと呼びます。曖昧な定義で抽象度が高い言葉ですが、そういうものと割り切ってください。その為、インスタンスはオブジェクトの1種と言えます。



4.2. オブジェクト指向プログラミングの記載方法

4.2.1. 活用事例と用語確認

実際の問題を解くために必要な知識に踏み込みます。飲食店の注文システムを開発するケースを考えます。これはメニューが並んでおり、個数を入力すると商品ごとの小計金額が算出されるシステムです。



各メニューの基本構造は同じなので、まずは設計書を作成し、そこから各メニューを量産します。さて、設計書を作成するために 1 つメニューを確認しましょう。設計書では「データ」と「処理」をまとめて開発するのでしたね。その為、データと処理の 2 観点で確認しましょう。

【データ】各メニューは“商品名”や“価格”といったデータを保持しています。オブジェクト指向では、このようなデータを**メンバ変数**と呼びます。メンバ変数は聞き慣れない言葉ですが、普段私たちが使っている変数と同じもので、クラス内で使われる変数は特別にメンバ変数と呼ばれるだけです。

【処理】各メニューは注文数に応じて金額を計算するというプログラムを保持しています。これを**メソッド**と呼びます。プログラムとほぼ同じ意味で、クラス内で使われるプログラムを特別にメソッドと呼びます。

よって、メニューを開発する際は、以下のようなメンバ変数やメソッドが含まれる設計書を作成し、そこから各メニューを量産して開発を進めていきます。具外的なコードを次の節で確認しましょう。



4.2.2. クラス型変数とインスタンスの生成

◆クラスの作成

まずは設計書、つまりクラスを作成します。今回は”Menu クラス”を作成します。クラス名の先頭文字は大文字で表現されます。実際のプログラミングではクラスの中身も自分でコードを書く必要がありますが、試験では問われないので具体的なコードは省略します。今回は、クラスの中にメニューの名前と値段を入れるメンバ変数 name,price、また注文の小計を計算するメソッド sum を定義します。

クラスの作成

```
Class Menu{
  ~クラスを定義するコード~
  ※試験には出題されない為省略
}
```

設計書 (Menuクラス)

実際の試験では以下のようにクラスの説明が問題文で記載されます。この説明を基に、メンバ変数やメソッドを整理して理解します。具体的な読み方は問題解説の章で説明します。

コンストラクタ		説明
PrioQueue()		空の優先度付きキューを生成する。
メソッド	戻り値	説明
enqueue(文字列型: s, 整数型: prio)	なし	優先度付きキューに、文字列 s を要素として、優先度 prio で追加する。
dequeue()	文字列型	優先度付きキューからキュー内で最も優先度の高い要素を取り出して返す。最も優先度の高い要素が複数あるときは、そのうちの最初に追加された要素を一つ取り出して返す。
size()	整数型	優先度付きキューに格納されている要素の個数を返す。

図 クラス PrioQueue の説明

引用：基本情報技術者試験 B試験サンプル問題 問8

設計書 Menu クラスを作ることが出来たら、この後はクラスからインスタンスを作成し、各メニューを作ります。インスタンス作成は以下の流れです。

- 1. クラス型変数を定義する
- 2. インスタンスを作成し、そのメモリアドレスをクラス型変数に格納する

◆クラス型変数

クラス型変数について説明します。前章で確認した通り、変数には型があり、変数宣言時に型を指定するのです。整数型の変数 “i” を宣言する際は、【整数型：i】と定義します。一方、クラス型変数 “currymenu” を宣言する方法は特殊で、【Menu：currymenu】と宣言します。

＜整数型変数の宣言＞

整数型：i

＜クラス型変数の宣言＞

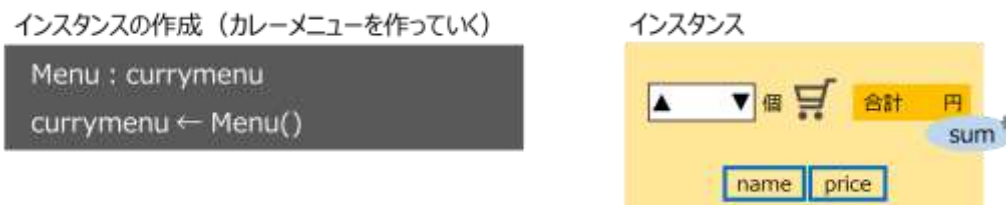
Menu：currymenu

このように、クラス型変数は **【クラス名：変数名】** と指定されるので、慣れないうちは分かりにくいです。クラス名は問題によって異なるので、宣言時の記載も問題毎に固有です。また、変数の名前は作成されるインスタンス名に関連することが多いです。今回はカレーのメニューを作成するためのインスタンスを作るので、変数名は currymenu となっています。

クラス型変数に格納される値は少し特殊です。整数型変数は整数が、論理型変数には論理値が格納されるのでしたね。ですが、**クラス型変数には生成されたインスタンスのメモリアドレスが格納されます。** メモリはコンピューターの中でデータを記憶しておく領域のことで、メモリアドレスが割り振られているのです。生成されたインスタンスはメモリのどこかに保存され、そのメモリアドレスがクラス型変数に格納されます。



次にインスタンスの作成方法を確認します。以下のコードを記載することでインスタンスが生成されます。



【クラス型変数 ← クラス名()】 という記載はインスタンスの生成を指示し、そのメモリアドレスを変数に格納するコードです。ただし、これだけではインスタンスが作成されただけで、中身のメンバ変数は空っぽのままなので、メンバ変数にデータを入れる命令も追加していきます。

インスタンスの作成（カレーメニューを作っていく）

```
Menu : currymenu  
currymenu ← Menu()  
currymenu.name ← “カレー”  
currymenu.price ← 800
```

インスタンス



この 4 行の処理で、インスタンスの作成及びメンバ変数へのデータ格納が完了します。処理を覚えることに意味は無いので、処理の内容を詳しく説明していきます。

1 行目 : Menu : currymenu

クラス型変数 “currymenu” を定義しています。

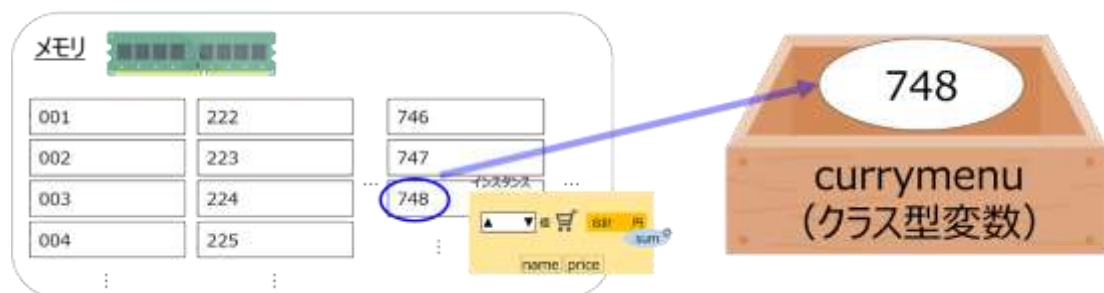
2 行目 : currymenu ← Menu()

Menu()でインスタンスが生成され、クラス型変数 currymenu にメモリアドレスが代入されています。ぱっと見、変数 currymenu に直接インスタンスが格納されているように見えますが、これは誤りです。



<誤ったクラス型変数のイメージ>

先ほど説明した通り、クラス型変数にはインスタンスのメモリアドレスが格納されます。インスタンスが生成されると、メモリのどこかにインスタンスが保存されます。クラス型変数には、このインスタンスが生成されたメモリアドレスが格納されています。データそのものではなく、データの場所情報が格納されるということです。



🔥 3 行目 : `currymenu.name ← “カレー”`

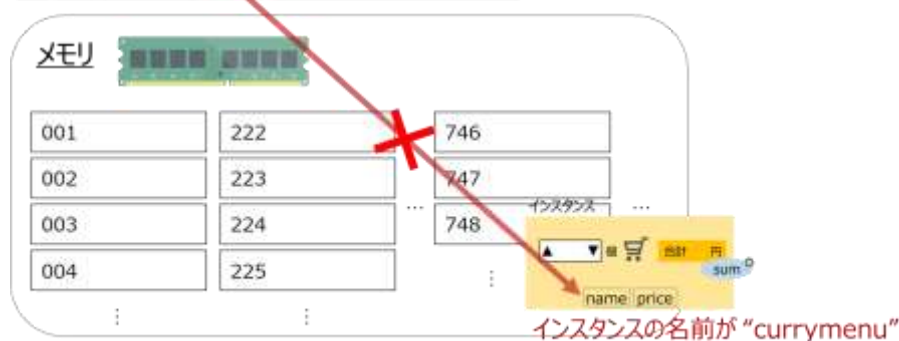
🔥 4 行目 : `currymenu.price ← 800`

この 2 つの処理でメンバ変数にデータが代入されます。「クラス型変数.メンバ変数 / クラス型変数.メソッド」で指定することで、クラス型変数内のメモリアドレスに格納されているインスタンスのメンバ変数もしくはメソッドへアクセスできます。これはメンバ変数もしくはメソッドの参照と呼び、「`.`」は参照を意味する演算子です。つまり、3 行目と 4 行目は、748 のメモリアドレスに格納されてるインスタンスのメンバ変数 `name`, `price` にアクセスし、それぞれに「カレー」「800」を代入しています。

混乱しやすい点として、インスタンスの名前が `currymenu` と思い、「`currymenu.name` は直接インスタンス内のメンバ変数 `name` にアクセスする」と勘違いする人が多いのですが、間違いです。

インスタンスの作成

```
Menu : currymenu
currymenu ← Menu()
currymenu.name ← “カレー”
currymenu.price ← 800
```



<誤った参照のイメージ>

あくまで、変数 `currymenu` にあるメモリアドレスを経由してインスタンスにアクセスするので、例えば変数に 222 が格納されている場合、222 番のメモリにアクセスしてエラーとなります。最初は違和感を感じるかもしれませんが、大切な考え方なので必ず理解しておきましょう。



4.2.3. コンストラクタの活用

これでカレーのメニューを作ることが出来たので、他のメニューを作成してみます。カレーメニューで実施したように、「①クラス型変数を定義②インスタンスを作成③メンバ変数にデータを格納」という処理を行います。

カレーメニュー作成



```
Menu : currymenu
currymenu ← Menu()
currymenu.name ← “カレー”
currymenu.price ← 800
```

サラダメニューの作成



```
Menu : saladmenu
saladmenu ← Menu()
saladmenu.name ← “サラダ”
saladmenu.price ← 300
```

グラタンメニューの作成



```
Menu : gratinmenu
gratinmenu ← Menu()
gratinmenu.name ← “グラタン”
gratinmenu.price ← 950
```

...

ですが、「③メンバ変数にデータを格納」について、全てのメニューでメニュー情報と価格を一々入れていくのは結構めんどくさいですね。そこで、これらデータがメソッドに自動入力される方法を学習します。

自動入力する方法は、クラスを定義したコードの中に“コンストラクタ”と呼ばれるメソッドを追加して、メンバ変数に引数を自動入力するようコードを書くことです。（コンストラクタのコードも試験で問われないので記載方法の説明は省略します。）

コンストラクタとはメソッドの特別版のことで、コンストラクタで定義した処理はインスタンス生成時に自動で実行されます。1つのクラスにつき、基本は1つのコンストラクタが定義され、コンストラクタとクラスの名称は同じです。クラス内にコンストラクタを記載することで、メソッドに加えコンストラクタも追加されます。

クラスの作成

```
Class Menu{
  ~クラスを定義するコード~
  ※試験には出題されない為省略

  ~メンバ変数に引数を自動入力する
  コンストラクタを定義するコード~
  ※試験には出題されない為省略
}
```

設計書 (Menuクラス)



コンストラクタ Menuが作成

メソッドの特別版であり、コンストラクタで定義した処理は
インスタンス生成に実行される (※名称はクラス名と一致)

<メソッドとコンストラクタの比較>

観点	メソッド	コンストラクタ
名称	自由に定義可能	クラス名と同じ
処理内容	自由に定義可能	自由に定義可能
実行タイミング	任意のタイミングで呼び出す	インスタンス生成時

例えば、以下のプログラムでは 2 行目でインスタンスを作成して変数カレーメニューに代入します。クラスにコンストラクタが定義されている場合、インスタンス生成のタイミングでコンストラクタが自動で起動します。メソッドを使いたい場合は、その後に「クラス型変数.メソッド名」と記載して処理を呼び出します。

```
Menu : currymenu
currymenu ← Menu()
currymenu.sum
```

— インスタンス生成時に勝手にコンストラクタの処理が起動

— メソッドの処理をするには参照する必要がある

まとめると以下ようになります。

＜クラスの構成要素＞

メンバ変数	データ
メソッド	処理
コンストラクタ	メソッドの特別版

＜インスタンスの構成要素＞

メンバ変数	データ
メソッド	処理

- ✓ クラスの構成要素は全て定義されているケースも、いずれかしか使わないケースもある
- ✓ クラスの構成要素の内容は、試験では表でまとめて説明される
- ✓ メンバ変数はクラスにあるということ以外、通常の変数と同様の為変数の型が存在
- ✓ メソッドやコンストラクタも、クラス内にあること以外は通常のプログラムと同様
- ✓ インスタンスはコンストラクタを保持しない

コンストラクタを使用することで、メニューを簡単に作ることが出来ます。例えば、クラス内に「インスタンス生成時に第一引数をメンバ変数 name に、第二引数をメンバ変数 price に格納する」というコンストラクタを定義した場合、インスタンス生成処理と同時にメンバ変数に値が代入されます。（以下の図のように、インスタンス生成時に()内で引数としてデータを定義することも出来ます。）



コンストラクタが無い場合と比較するとコンストラクタの有用性がわかります。コンストラクタを定義しないと、インスタンスを作った後に一々名前と価格をメンバ変数に代入していましたが、一度クラス内にコンストラクタを用意してしまえば、インスタンスを作成するだけでメニューが勝手に生成されます。



4.3. オブジェクト指向プログラミングのまとめと練習

オブジェクト指向の重要な点を最後にまとめておきます。最初は混乱しますが、分からなくなったら都度この基本原則に立ち返り、身に着けていきましょう。

重要事項	説明
クラス	共通事項をまとめた設計書で、以下の全て、もしくはいずれかを含む ・メンバ変数：クラス内の変数 ・メソッド：クラス内の処理 ・コンストラクタ：インスタンス生成時に実行されるメソッドの特別版でクラスと同じ名前
クラス型変数	インスタンスのメモリアドレスを格納する変数で、以下のコードで宣言する ・クラス名：変数名
インスタンス	クラスから作成された具体的な実体で、以下の内容を含む ※コンストラクタはインスタンス生成時に処理が実行されて役割が終わる為、インスタンスはコンストラクタを保持しない ・メンバ変数：クラス内の変数 ・メソッド：クラス内の処理
インスタンス生成	【クラス型変数 ← クラス名（第 1 引数, 第 2 引数, …）】でインスタンスが生成され、クラス型変数にインスタンスのメモリアドレスが格納される ※引数が無いこともある ※コンストラクタがクラスで定義されている場合、このタイミングで勝手に起動する
メンバ変数の参照 メソッドの参照	以下のコードでインスタンス内のメンバ変数やメソッドにアクセスできる ・クラス型変数.メンバ変数 ・クラス型変数.メソッド

最後に練習問題です。CreatAccount プログラムを実行した際、口座残高に残っている金額を求めましょう。動画で解説を行っているので、適宜本教材にメモしておいてください。

account クラス

メンバ変数	型	説明
name	文字列型	口座名義を格納するメンバ変数
balance	整数型	口座残高を格納するメンバ変数

メソッド	戻り値	説明
deposit (整数型 : n)	なし	balanceに格納されている口座残高に n を増やす
withdraw (整数型 : n)	なし	balanceに格納されている口座残高から n を減らす

コンストラクタ	説明
Account (文字型 : my_name, 整数型 : my_balance)	引数 my_name、my_balanceでメンバ変数name, balanceを初期化する

○CreateAccount (文字型 : my_name, 整数型 : my_balance)

```
Account : my_account
```

```
my_account ← Account (my_name, my_balance)
```

```
while my_account.balance ≤ 3000
```

```
    my_account.deposit (700)
```

```
endwhile
```

```
my_account.withdraw (200)
```

5. サンプル問題解説

ここからサンプル問題の解説に移ります。補足説明以外は問題のみ記載するので、詳細な解説は動画を確認し、適宜メモを取ってください。

5.1. 1～5 問目

◆1 問目/26 問（問題）

ある施設の入場料は、0 歳から 3 歳までは 100 円、4 歳から 9 歳までは 300 円、10 歳以上は 500 円である。関数 fee は、年齢を表す 0 以上の整数を引数として受け取り、入場料を返す。

〔プログラム〕

```
○整数型: fee(整数型: age)
  整数型: ret
  if (age が 3 以下)
    ret ← 100
  elseif (  )
    ret ← 300
  else
    ret ← 500
  endif
  return ret
```

解答群

- ア (age が 4 以上) and (age が 9 より小さい)
- イ (age が 4 と等しい) or (age が 9 と等しい)
- ウ (age が 4 より大きい) and (age が 9 以下)
- エ age が 4 以上
- オ age が 4 より大きい
- カ age が 9 以下
- キ age が 9 より小さい

引用：基本情報技術者試験 B試験サンプル問題 問1

<  回答用紙 >

◆1 問目/26 問 (解説メモ用)

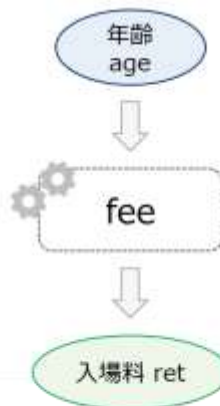
ある施設の入場料は、0 歳から 3 歳までは 100 円、4 歳から 9 歳までは 300 円、10 歳以上は 500 円である。関数 fee は、年齢を表す 0 以上の整数を引数として受け取り、入場料を返す。

〔プログラム〕

入るモノ

```
○整数型: fee(整数型: age)
整数型: ret
if (age が 3 以下)
  ret ← 100
elseif ( )
  ret ← 300
else
  ret ← 500
endif
return ret
```

出てくるモノ



1. 処理全体像の把握
2. アルゴリズムを階層的に理解
3. 問題文と疑似言語を紐づける
4. トレースして挙動をイメージする

引用：基本情報技術者試験 B試験サンプル問題 問1

ある施設の入場料は、0 歳から 3 歳までは 100 円、4 歳から 9 歳までは 300 円、10 歳以上は 500 円である。関数 fee は、年齢を表す 0 以上の整数を引数として受け取り、入場料を返す。

1. 処理全体像の把握
2. アルゴリズムを階層的に理解
3. 問題文と疑似言語を紐づける
4. トレースして挙動をイメージする

〔プログラム〕

```
○整数型: fee(整数型: age)
整数型: ret
if (age が 3 以下)
  ret ← 100
elseif ( )
  ret ← 300
else
  ret ← 500
endif
return ret
```

解答群

- ア (age が 4 以上) and (age が 9 より小さい)
- イ (age が 4 と等しい) or (age が 9 と等しい)
- ウ (age が 4 より大きい) and (age が 9 以下)
- エ age が 4 以上
- オ age が 4 より大きい
- カ age が 9 以下
- キ age が 9 より小さい

if 文では条件にヒットしたら処理を行って if 文から抜け出す
→ age が 4 歳以上の場合のみ 2 つ目の分岐に来る

空欄には age が 9 歳以下の条件を記載

引用：基本情報技術者試験 B試験サンプル問題 問1

◆2 問目/26 問（問題）

問2 次のプログラム中の と に入れる正しい答えの組合せを，解答群の中から選べ。ここで，配列の要素番号は1から始まる。

次のプログラムは，整数型の配列arrayの要素の並びを逆順にする。

〔プログラム〕

```
整数型の配列: array ← {1, 2, 3, 4, 5}
整数型: right, left
整数型: tmp

for (left を 1 から (arrayの要素数 ÷ 2 の商) まで 1 ずつ増やす)
  right ← 
  tmp ← array[right]
  array[right] ← array[left]
   ← tmp
endfor
```

解答群

	a	b
ア	array の要素数 - left	array[left]
イ	array の要素数 - left	array[right]
ウ	array の要素数 - left + 1	array[left]
エ	array の要素数 - left + 1	array[right]

引用：基本情報技術者試験 B試験サンプル問題 問2

<  回答用紙 >

問2 次のプログラム中の と に入れる正しい答えの組合せを、解答群の中から選べ。ここで、配列の要素番号は1から始まる。

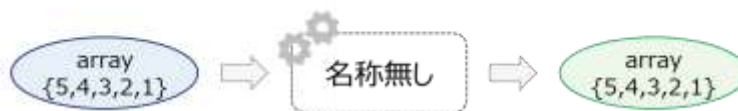
次のプログラムは、整数型の配列arrayの要素の並びを逆順にする。

```
整数型の配列: array ← (1, 2, 3, 4, 5)
整数型: right, left
整数型: tmp
```

```

right ← a
tmp ← array[right]
array[right] ← array[left]
b ← tmp
endfor

```



1	2	3	4	5
1	2	3	4	5
right	left	tmp		

[illegible]

◆3 問目/26 問（問題）

手続 `append` は、引数で与えられた文字を単方向リストに追加する手続である。単方向リストの各要素は、クラス `ListElement` を用いて表現する。クラス `ListElement` の説明を図に示す。`ListElement` 型の変数はクラス `ListElement` のインスタンスの参照を格納するものとする。大域変数 `listHead` は、単方向リストの先頭の要素の参照を格納する。リストが空のときは、`listHead` は未定義である。

メンバ変数	型	説明
<code>val</code>	文字型	リストに格納する文字。
<code>next</code>	<code>ListElement</code>	リストの次の文字を保持するインスタンスの参照。初期状態は未定義である。

コンストラクタ	説明
<code>ListElement(文字型: qVal)</code>	引数 <code>qVal</code> でメンバ変数 <code>val</code> を初期化する。

図 クラス `ListElement` の説明

〔プログラム〕

大域: `ListElement: listHead` ← 未定義の値

```
○append(文字型: qVal)
  ListElement: prev, curr
  curr ← ListElement(qVal)
  if (listHead が a)
    listHead ← curr
  else
    prev ← listHead
    while (prev.next が 未定義でない)
      prev ← prev.next
    endwhile
    prev.next ← b
  endif
```

解答群

	a	b
ア	未定義	<code>curr</code>
イ	未定義	<code>curr.next</code>
ウ	未定義	<code>listHead</code>
エ	未定義でない	<code>curr</code>
オ	未定義でない	<code>curr.next</code>
カ	未定義でない	<code>listHead</code>

引用：基本情報技術者試験 B試験サンプル問題 問3

<  回答用紙 >

< 📄 回答用紙 >

◆3 問目/26 問（解説メモ用）

【プログラム】

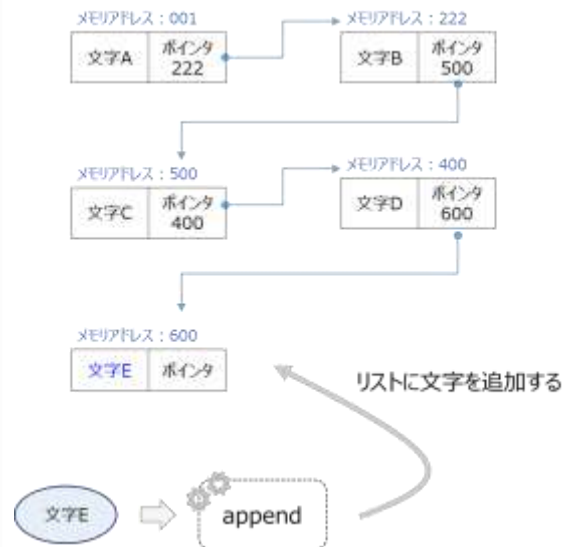
大域: ListElement: listHead ← 未定義の値

```

○append(文字型: qVal)
  ListElement: prev, curr
  curr ← ListElement(qVal)
  if (listHead が )
    listHead ← curr
  else
    prev ← listHead
    while (prev.next が 未定義でない)
      prev ← prev.next
    endwhile
    prev.next ← 
  endif
endif

```

引用：基本情報技術者試験 B試験サンプル問題 問3



ListHead

Prev

curr

インスタンス S

- メンバ変数
 - ✓ 文字型変数 val :
 - ✓ クラス型変数 next :

インスタンス t

- メンバ変数
 - ✓ 文字型変数 val :
 - ✓ クラス型変数 next :

インスタンス a

- メンバ変数
 - ✓ 文字型変数 val :
 - ✓ クラス型変数 next :

インスタンス r

- メンバ変数
 - ✓ 文字型変数 val :
 - ✓ クラス型変数 next :

◆4 問目/26 問 (問題)

問4 次の記述中の ～ に入れる正しい答えの組合せを、解答群の中から選べ。ここで、配列の要素番号は1から始まる。

要素の多くが0の行列を疎行列という。次のプログラムは、二次元配列に格納された行列のデータ量を削減するために、疎行列の格納に適したデータ構造に変換する。

関数 transformSparseMatrix は、引数 matrix で二次元配列として与えられた行列を、整数型配列の配列に変換して返す。関数 transformSparseMatrix を transformSparseMatrix({{3, 0, 0, 0, 0}, {0, 2, 2, 0, 0}, {0, 0, 0, 1, 3}, {0, 0, 0, 2, 0}, {0, 0, 0, 0, 1}})として呼び出したときの戻り値は、{{ }, { }, { }}である。

[プログラム]

```
○整数型配列の配列: transformSparseMatrix(整数型の二次元配列: matrix)
整数型: i, j
整数型配列の配列: sparseMatrix
sparseMatrix ← {{}, {}, {}} /* 要素数0の配列を三つ要素にもつ配列 */
for (i を 1 から matrixの行数 まで 1 ずつ増やす)
  for (j を 1 から matrixの列数 まで 1 ずつ増やす)
    if (matrix[i, j] が 0 でない)
      sparseMatrix[1]の末尾 に iの値 を追加する
      sparseMatrix[2]の末尾 に jの値 を追加する
      sparseMatrix[3]の末尾 に matrix[i, j]の値 を追加する
    endif
  endfor
endfor
return sparseMatrix
```

解答群

	a	b	c
ア	1, 2, 2, 3, 3, 4, 5	1, 2, 3, 4, 5, 4, 5	3, 2, 2, 1, 2, 3, 1
イ	1, 2, 2, 3, 3, 4, 5	1, 2, 3, 4, 5, 4, 5	3, 2, 2, 1, 3, 2, 1
ウ	1, 2, 3, 4, 5, 4, 5	1, 2, 2, 3, 3, 4, 5	3, 2, 2, 1, 2, 3, 1
エ	1, 2, 3, 4, 5, 4, 5	1, 2, 2, 3, 3, 4, 5	3, 2, 2, 1, 3, 2, 1

< 📄 回答用紙 >

◆4 問目/26 問（解説メモ用）

問4 次の記述中の ～ に入れる正しい答えの組合せを、解答群の中から選べ。ここで、配列の要素番号は1から始まる。

要素の多くが0の行列を疎行列という。次のプログラムは、二次元配列に格納された行列のデータ量を削減するために、疎行列の格納に適したデータ構造に変換する。

関数 transformSparseMatrix は、引数 matrix で二次元配列として与えられた行列を、整数型配列の配列に変換して返す。関数 transformSparseMatrix を `transformSparseMatrix({{3, 0, 0, 0, 0}, {0, 2, 2, 0, 0}, {0, 0, 0, 1, 3}, {0, 0, 0, 2, 0}, {0, 0, 0, 0, 1}})` として呼び出したときの戻り値は、`{{ , { , {  }}` である。



○整数型配列の配列: transformSparseMatrix(整数型の二次元配列: matrix)
 整数型: i, j
 整数型配列の配列: sparseMatrix
 sparseMatrix ← {{}, {}, {}} /* 要素数0の配列を三つ要素にもつ配列 */
 for (i を 1 から matrixの行数 まで 1 ずつ増やす)
 for (j を 1 から matrixの列数 まで 1 ずつ増やす)
 if (matrix[i, j] が 0 でない)
 sparseMatrix[1]の末尾 に iの値 を追加する
 sparseMatrix[2]の末尾 に jの値 を追加する
 sparseMatrix[3]の末尾 に matrix[i, j]の値 を追加する
 endif
 endfor
 endfor
 return sparseMatrix

配列 Matrix

	1	2	3	4	5
1	3	0	0	0	0
2	0	2	2	0	0
3	0	0	0	1	3
4	0	0	0	2	0
5	0	0	0	0	1

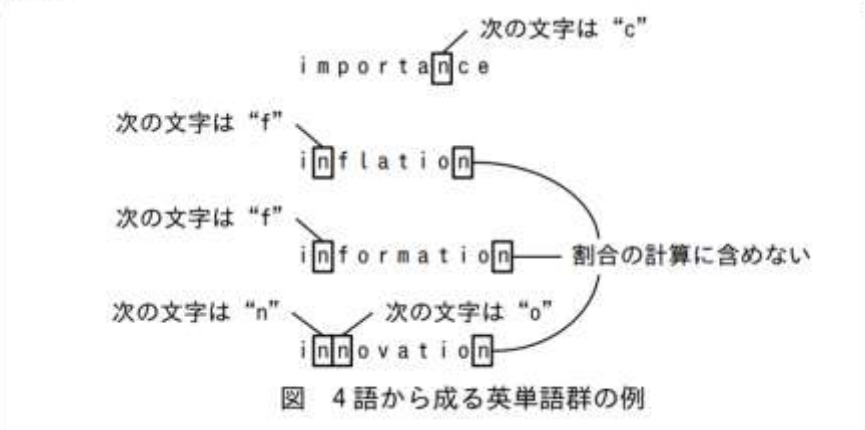
処理	i	j	matrix[i,j]

配列 sparseMatrix

	1	2	3	4	5	6	7
1							
2							
3							

◆5 問目/26 問（問題）

任意の異なる 2 文字を c_1 , c_2 とするとき、英単語群に含まれる英単語において、 c_1 の次に c_2 が出現する割合を求めるプログラムである。英単語は、英小文字だけから成る。英単語の末尾の文字が c_1 である場合、その箇所は割合の計算に含めない。例えば、図に示す 4 語の英単語 “importance”, “inflation”, “information”, “innovation” から成る英単語群において、 c_1 を “n”, c_2 を “f” とする。英単語の末尾の文字以外に “n” は五つあり、そのうち次の文字が “f” であるものは二つである。したがって、求める割合は、 $2 \div 5 = 0.4$ である。 c_1 と c_2 の並びが一度も出現しない場合、 c_1 の出現回数によらず割合を 0 と定義する。



プログラムにおいて、英単語群は Words 型の大域変数 words に格納されている。クラス Words のメソッドの説明を、表に示す。本問において、文字列に対する演算子 “+” は、文字列の連結を表す。また、整数に対する演算子 “÷” は、実数として計算する。

表 クラス Words のメソッドの説明

メソッド	戻り値	説明
freq(文字列型: str)	整数型	英単語群中の文字列 str の出現回数を返す。
freqE(文字列型: str)	整数型	英単語群の中で、文字列 str で終わる英単語の数を返す。

〔プログラム〕

```
大域: Words: words /* 英単語群が格納されている */
```

```
/* c1の次にc2が出現する割合を返す */
```

```
○実数型: prob(文字型: c1, 文字型: c2)
```

```
文字列型: s1 ← c1の1文字だけから成る文字列
```

```
文字列型: s2 ← c2の1文字だけから成る文字列
```

```
if (words.freq(s1 + s2) が 0 より大きい)
```

```
    return 
```

```
else
```

```
    return 0
```

```
endif
```

解答群

ア $(\text{words.freq}(s1) - \text{words.freqE}(s1)) \div \text{words.freq}(s1 + s2)$

イ $(\text{words.freq}(s2) - \text{words.freqE}(s2)) \div \text{words.freq}(s1 + s2)$

ウ $\text{words.freq}(s1 + s2) \div (\text{words.freq}(s1) - \text{words.freqE}(s1))$

エ $\text{words.freq}(s1 + s2) \div (\text{words.freq}(s2) - \text{words.freqE}(s2))$

引用: 基本情報技術者試験 B試験サンプル問題 問5

<  回答用紙 >

◆5 問目/26 問（解説メモ用）

（例）英単語の中で n の後に f が来る割合

i m p o r t a n c e

i n f l a t i o n

i n f o r m a t i o n

i n n o v a t i o n

次の文字が c2 である c1 の数

末尾以外に出現する c1 の

表 クラス Words のメソッドの説明

メソッド	戻り値	説明
freq(文字列型: str)	整数型	英単語群中の文字列 str の出現回数を返す。
freqE(文字列型: str)	整数型	英単語群の中で、文字列 str で終わる英単語の数を返す。

★メソッドはクラスの中にあるというだけで、プログラムには変わらないのでいつもの3点セットを確認

【どんなデータを入れる】

【どんな処理を行う】

【どんなデータが返ってくる】

クラスの構成要素

- ・メンバ変数（データ）：英単語が格納
- ・メソッド（処理）：freq, freqE
- ・コンストラクタ（処理の特別版）：定義なし



【プログラム】

大域: Words: words /* 英単語群が格納されている */

/* c1の次にc2が出現する割合を返す */

○実数型: prob(文字型: c1, 文字型: c2)

文字列型: s1 ← c1の1文字だけから成る文字列

文字列型: s2 ← c2の1文字だけから成る文字列

if (words.freq(s1 + s2) が 0 より大きい)

return words.freq(s1 + s2) / (words.freq(s1) - words.freqE(s1))

else

return 0

endif

★c1 の次に c2 が出現する割合を示す以下の式が入ればよい

次の文字が c2 である c1 の数

末尾以外に出現する c1 の数

【分子】：次の文字が c2 である c1 の数

= 文字列 "c1c2" の数

words.freq(s1 + s2)

【分母】：末尾以外に出現する c1 の数

= c1の全体数 - 末尾に登場するc1の数

words.freq(s1)

words.freqE(s1)

◆1～5 問目 まとめ（気づきやミスを追加してメモしておきましょう！）

問題	タイプ	問題概要	ポイント（気づき/解法/得たスキルなど）
1	穴埋め	if 文を用いて年齢ごとの入場料を算出するプログラムを作成	- 問題文及びプログラムより、引数と戻り値の変数とそれが意味するものを整理して考える - if文の条件分岐を活用してプログラムを作成する
2	穴埋め	配列要素の並び替えプログラムを作成	- 先にプログラムの全体像を理解することで、トレースしながら処理内容を予測できる - 変数 tmp を用いて要素の入れ替え処理を行う
3	穴埋め	オブジェクト指向を用いた単方向リストの要素追加処理を行うプログラムを作成	- オブジェクト指向では「クラス・メソッド・コンストラクタ」を整理して考える - 変数や配列の遷移が激しくない場合は実際の配列や変数の箱を用意して処理トラッキングする
4	処理結果	疎行列のデータ量削減プログラムの処理結果	2次元配列が登場する処理で for文の入れ子がある場合、配列の要素を1つずつチェックする処理である可能性が高い - トレースにより処理の挙動が掴めたら、それを一般化させて時短する
5	穴埋め	英単語群の中の登場文字列割合を算出するプログラムを作成	- 問題文にある具体的な処理は必ず確認し、プログラムの挙動の解像度を高める - メソッドもプログラムと同様「何を入れる？どんな処理？何が出てくる？」の3点を確認する

5.2. 6～10 問目

◆6 問目/26 問 (問題)

問1 次の記述中の に入れる正しい答えを、解答群の中から選べ。

プログラムを実行すると、“ ”と出力される。

〔プログラム〕

整数型: $x \leftarrow 1$

整数型: $y \leftarrow 2$

整数型: $z \leftarrow 3$

$x \leftarrow y$

$y \leftarrow z$

$z \leftarrow x$

y の値 と z の値 をこの順にコンマ区切りで出力する

解答群

ア 1,2

イ 1,3

ウ 2,1

エ 2,3

オ 3,1

カ 3,2

引用：基本情報技術者試験 B試験サンプル問題 問1

<  回答用紙 >

◆6 問目/26 問（解説メモ用）

問 1 次の記述中の に入れる正しい答えを、解答群の中から選べ。

プログラムを実行すると、“”と出力される。

〔プログラム〕

```
整数型: x ← 1
整数型: y ← 2
整数型: z ← 3
x ← y
y ← z
z ← x
yの値 と zの値 をこの順にコンマ区切りで出力する
```

解答群

ア 1, 2	イ 1, 3	ウ 2, 1
エ 2, 3	オ 3, 1	カ 3, 2

引用：基本情報技術者試験 B試験サンプル問題 問1

処理	x	y	z

◆7 問目/26 問 (問題)

問2 次のプログラム中の a ～ c に入れる正しい答えの組合せを、
解答群の中から選べ。

関数 fizzBuzz は、引数で与えられた値が、3 で割り切れて 5 で割り切れない場合は “3 で割り切れる” を、5 で割り切れて 3 で割り切れない場合は “5 で割り切れる” を、3 と 5 で割り切れる場合は “3 と 5 で割り切れる” を返す。それ以外の場合は “3 でも 5 でも割り切れない” を返す。

〔プログラム〕

```
○文字列型: fizzBuzz(整数型: num)
文字列型: result
if (num が a で割り切れる)
    result ← "a で割り切れる"
elseif (num が b で割り切れる)
    result ← "b で割り切れる"
elseif (num が c で割り切れる)
    result ← "c で割り切れる"
else
    result ← "3 でも 5 でも割り切れない"
endif
return result
```

解答群

	a	b	c
ア	3	3と5	5
イ	3	5	3と5
ウ	3と5	3	5
エ	5	3	3と5
オ	5	3と5	3

引用：基本情報技術者試験 B試験サンプル問題 問2

<  回答用紙 >

◆7 問目/26 問 (解説メモ用)

問2 次のプログラム中の ～ に入れる正しい答えの組合せを、
解答群の中から選べ。

関数 fizzBuzz は、引数で与えられた値が、3 で割り切れて 5 で割り切れない場合は “3 で割り切れる” を、5 で割り切れて 3 で割り切れない場合は “5 で割り切れる” を、3 と 5 で割り切れる場合は “3 と 5 で割り切れる” を返す。それ以外の場合は “3 でも 5 でも割り切れない” を返す。

〔プログラム〕

○文字列型: fizzBuzz(整数型: num)

文字列型: result

if (num が で割り切れる)

result ← “ で割り切れる”

elseif (num が で割り切れる)

result ← “ で割り切れる”

elseif (num が で割り切れる)

result ← “ で割り切れる”

else

result ← “3 でも 5 でも割り切れない”

endif

return result

◆8 問目/26 問 (問題)

問3 次の記述中の に入れる正しい答えを、解答群の中から選べ。ここで、配列の要素番号は1から始まる。

関数 `makeNewArray` は、要素数 2 以上の整数型の配列を引数にとり、整数型の配列を返す関数である。関数 `makeNewArray` を `makeNewArray({3, 2, 1, 6, 5, 4})` として呼び出したとき、戻り値の配列の要素番号 5 の値は となる。

〔プログラム〕

```
○ 整数型の配列: makeNewArray(整数型の配列: in)
  整数型の配列: out ← {} // 要素数0の配列
  整数型: i, tail
  outの末尾に in[1]の値を追加する
  for (i を 2 から inの要素数 まで 1 ずつ増やす)
    tail ← out[outの要素数]
    outの末尾に (tail + in[i]) の結果を追加する
  endfor
  return out
```

解答群

ア 5 イ 6 ウ 9 エ 11 オ 12
カ 17 キ 21

引用：基本情報技術者試験 B試験サンプル問題 問3

<  回答用紙 >

◆8 問目/26 問 (解説メモ用)

問3 次の記述中の に入れる正しい答えを、解答群の中から選べ。ここで、配列の要素番号は1から始まる。

関数 `makeNewArray` は、要素数 2 以上の整数型の配列を引数にとり、整数型の配列を返す関数である。関数 `makeNewArray` を `makeNewArray({3, 2, 1, 6, 5, 4})` として呼び出したとき、戻り値の配列の要素番号 5 の値は となる。

〔プログラム〕

```

○整数型の配列: makeNewArray(整数型の配列: in) 引数 (入力データ)
  整数型の配列: out ← {} // 要素数0の配列
  整数型: i, tail
  outの末尾 に in[1]の値 を追加する
  for (i を 2 から inの要素数 まで 1 ずつ増やす)
    tail ← out[outの要素数]
    outの末尾 に (tail + in[i]) の結果を追加する
  endfor
  return out 戻り値 (出力データ)

```

[illegible]

配列 in

1	2	3	4	5	6

配列 out

1	2	3	4	5	6

＜参考：配列out をトレース表で管理する場合＞

[illegible]

◆9 問目/26 問 (問題)

問4 次のプログラム中の ～ に入れる正しい答えの組合せを、
解答群の中から選べ。

関数 gcd は、引数で与えられた二つの正の整数 num1 と num2 の最大公約数を、次の(1)～(3)の性質を利用して求める。

- (1) num1 と num2 が等しいとき、num1 と num2 の最大公約数は num1 である。
- (2) num1 が num2 より大きいとき、num1 と num2 の最大公約数は、(num1 - num2) と num2 の最大公約数と等しい。
- (3) num2 が num1 より大きいとき、num1 と num2 の最大公約数は、(num2 - num1) と num1 の最大公約数と等しい。

〔プログラム〕

```
○整数型: gcd(整数型: num1, 整数型: num2)
整数型: x ← num1
整数型: y ← num2

if (  )
    x ← x - y
else
    y ← y - x
endif

return x
```

解答群

	a	b	c
ア	if (x ≠ y)	x < y	endif
イ	if (x ≠ y)	x > y	endif
ウ	while (x ≠ y)	x < y	endwhile
エ	while (x ≠ y)	x > y	endwhile

引用：基本情報技術者試験 B試験サンプル問題 問4

<  回答用紙 >

◆10 問目/26 問 (問題)

問5 次のプログラム中の に入れる正しい答えを、解答群の中から選べ。

関数 calc は、正の実数 x と y を受け取り、 $\sqrt{x^2+y^2}$ の計算結果を返す。関数 calc が使う関数 pow は、第 1 引数として正の実数 a を、第 2 引数として実数 b を受け取り、 a の b 乗の値を実数型で返す。

〔プログラム〕

```
○実数型: calc(実数型: x, 実数型: y)  
  return 
```

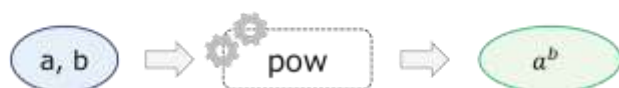
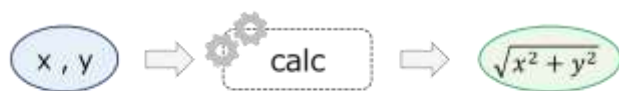
解答群

- ア $(\text{pow}(x, 2) + \text{pow}(y, 2)) \div \text{pow}(2, 0.5)$
- イ $(\text{pow}(x, 2) + \text{pow}(y, 2)) \div \text{pow}(x, y)$
- ウ $\text{pow}(2, \text{pow}(x, 0.5)) + \text{pow}(2, \text{pow}(y, 0.5))$
- エ $\text{pow}(\text{pow}(\text{pow}(2, x), y), 0.5)$
- オ $\text{pow}(\text{pow}(x, 2) + \text{pow}(y, 2), 0.5)$
- カ $\text{pow}(x, 2) \times \text{pow}(y, 2) \div \text{pow}(x, y)$
- キ $\text{pow}(x, y) \div \text{pow}(2, 0.5)$

引用：基本情報技術者試験 B試験サンプル問題 問5

<  回答用紙 >

◆ 10 問目/26 問 (解説メモ用)



$$\sqrt{x^2 + y^2} = (x^2 + y^2)^{0.5}$$

The expression is annotated with a blue 'a' above $x^2 + y^2$ and a red 'b' above 0.5.

$$= \text{pow}(x^2 + y^2, 0.5)$$

$$\begin{array}{cc} \downarrow & \downarrow \\ \text{pow}(x, 2) & \text{pow}(y, 2) \end{array}$$

◆6～10 問目 まとめ（気づきやミスを追加してメモしておきましょう！）

問題	タイプ	問題概要	ポイント（気づき/解法/得たスキルなど）
6	処理結果	変数への値代入処理	特になし（値の代入が理解出来れば解ける）
7	穴埋め	if文を使用し、条件に応じて引数の特性を出力するプログラムの作成	- プログラムを階層的に理解した上問題文と紐づける - if 文の特性（条件を満たさない場合は次の処理に移り、満たす場合は処理から抜け出す）を活用してプログラムを作成している為、問題文の条件とプログラム上の条件が見かけ上は一致しなかった - 具体例を作成してトレースを行う
8	処理結果	引数配列の累積和を配列として出力するプログラムの結果	特になし（配列を用いたデータの取り扱いが理解できていれば簡単に解ける）
9	穴埋め	最大公約数を求めるプログラムを作成	- 問題文の説明（今回は最大公約数を算出する方法）をプログラムに落とし込む - if文とwhile文はともに条件が true の時に次に処理に進むので、条件が同じ場合は、処理を1回だけ行うか複数回繰り返すかの違いしかない
10	穴埋め	関数を用いて平方根を出力するプログラムを作成	関数の呼び出しも「引数・出力」を整理して、意図する処理にうまく当てはめる

5.3. 11～15 問目

◆11 問目/26 問 (問題)

問6 次のプログラム中の に入れる正しい答えを、解答群の中から選べ。

関数 rev は 8 ビット型の引数 byte を受け取り、ビットの並びを逆にした値を返す。
例えば、関数 rev を rev(01001011) として呼び出すと、戻り値は 11010010 となる。

なお、演算子 $\wedge$ はビット単位の論理積、演算子 $\vee$ はビット単位の論理和、演算子 $\gg$ は論理右シフト、演算子 $\ll$ は論理左シフトを表す。例えば、value $\gg$ n は value の値を n ビットだけ右に論理シフトし、value $\ll$ n は value の値を n ビットだけ左に論理シフトする。

【プログラム】

```
○8ビット型: rev(8ビット型: byte)
  8ビット型: rbyte ← byte
  8ビット型: r ← 00000000
  整数型: i
  for (i を 1 から 8 まで 1 ずつ増やす)
    
  endfor
  return r
```

解答群

- ア $r \leftarrow (r \ll 1) \vee (rbyte \wedge 00000001)$
rbyte $\leftarrow rbyte \gg 1$
- イ $r \leftarrow (r \ll 7) \vee (rbyte \wedge 00000001)$
rbyte $\leftarrow rbyte \gg 7$
- ウ $r \leftarrow (rbyte \ll 1) \vee (rbyte \gg 7)$
rbyte $\leftarrow r$
- エ $r \leftarrow (rbyte \gg 1) \vee (rbyte \ll 7)$
rbyte $\leftarrow r$

引用：基本情報技術者試験 B試験サンプル問題 問6

<  回答用紙 >

◆11 問目/26 問（解説メモ用）

論理シフト：ビット列を左右にずらすこと

（例）01001011の2ビット左シフト



nビット左シフトすると
末尾nビットが必ず0

末尾は1との論理積：rbyteが1なら1、0なら0（rbyteの値そのまま）

rbyte	0	1	0	0	1	0	1	1
$\wedge$ (論理積)	0	0	0	0	0	0	0	1
	0	0	0	0	0	0	0	1

解答群

ア $r \leftarrow (r \ll 1) \vee (rbyte \wedge 00000001)$
rbyte $\leftarrow rbyte \gg 1$

イ $r \leftarrow (r \ll 7) \vee (rbyte \wedge 00000001)$
rbyte $\leftarrow rbyte \gg 7$

ウ $r \leftarrow (rbyte \ll 1) \vee (rbyte \gg 7)$
rbyte $\leftarrow r$

エ $r \leftarrow (rbyte \gg 1) \vee (rbyte \ll 7)$
rbyte $\leftarrow r$

解答群

ア $r \leftarrow (r \ll 1) \vee (rbyte \wedge 00000001)$
rbyte $\leftarrow rbyte \gg 1$

イ $r \leftarrow (r \ll 7) \vee (rbyte \wedge 00000001)$
rbyte $\leftarrow rbyte \gg 7$

ウ $r \leftarrow (rbyte \ll 1) \vee (rbyte \gg 7)$
rbyte $\leftarrow r$

エ $r \leftarrow (rbyte \gg 1) \vee (rbyte \ll 7)$
rbyte $\leftarrow r$

	0	0	0	0	0	0	0	0
$\vee$ (論理和)	0	0	0	0	0	0	0	1
変数 r	0	0	0	0	0	0	0	1

必ず rbyte の末尾

解答群

ア $r \leftarrow (r \ll 1) \vee (rbyte \wedge 00000001)$
rbyte $\leftarrow rbyte \gg 1$

イ $r \leftarrow (r \ll 7) \vee (rbyte \wedge 00000001)$
rbyte $\leftarrow rbyte \gg 7$

ウ $r \leftarrow (rbyte \ll 1) \vee (rbyte \gg 7)$
rbyte $\leftarrow r$

エ $r \leftarrow (rbyte \gg 1) \vee (rbyte \ll 7)$
rbyte $\leftarrow r$

rに格納した
値がはみ出す

変数 r	0	0	0	0	0	0	0	1
変数 rbyte	0	1	0	0	1	0	1	1
変数 rbyte	0	1	1	0	0	1	0	1

末尾が1つだけずれるので
1つずらしてrに格納できる

◆12 問目/26 問 (問題)

問7 次のプログラム中の に入れる正しい答えを、解答群の中から選べ。

関数 `factorial` は非負の整数 n を引数にとり、その階乗を返す関数である。非負の整数 n の階乗は n が 0 のときに 1 になり、それ以外の場合は 1 から n までの整数を全て掛け合わせた数となる。

〔プログラム〕

```
○整数型: factorial(整数型: n)
  if (n == 0)
    return 1
  endif
  return 
```

解答群

- | | |
|--|--------------------------------------|
| ア $(n - 1) \times \text{factorial}(n)$ | イ $\text{factorial}(n - 1)$ |
| ウ n | エ $n \times (n - 1)$ |
| オ $n \times \text{factorial}(1)$ | カ $n \times \text{factorial}(n - 1)$ |

引用：基本情報技術者試験 B試験サンプル問題 問7

<  回答用紙 >

◆12 問目/26 問（解説メモ用）

問7 次のプログラム中の に入れる正しい答えを、解答群の中から選べ。

関数 factorial は非負の整数 n を引数にとり、その階乗を返す関数である。非負の整数 n の階乗は n が 0 のときに 1 になり、それ以外の場合は 1 から n までの整数を全て掛け合わせた数となる。

〔プログラム〕

```
○ 整数型: factorial(整数型: n)
if (n == 0)
    return 1
endif
return  
```

○ プログラム sample

```
for 条件
-----
-----
-----
endfor
return xx
```

○ プログラム sample

```
for 条件
-----
return xx
-----
endfor
```

→ return の処理を行うので
プログラムが終了する

解答群

- | | |
|--|--------------------------------------|
| ア $(n - 1) \times \text{factorial}(n)$ | イ $\text{factorial}(n - 1)$ |
| ウ n | エ $n \times (n - 1)$ |
| オ $n \times \text{factorial}(1)$ | カ $n \times \text{factorial}(n - 1)$ |

引用：基本情報技術者試験 B試験サンプル問題 問7

選択肢（ア）

$\text{factorial}(4) = (4 - 1) \times \text{factorial}(4)$

↓

$(4 - 1) \times \text{factorial}(4)$

↓

$(4 - 1) \times \text{factorial}(4)$

⋮

✗

選択肢（イ）

$\text{factorial}(4) = \text{factorial}(4-1)$

↓

$\text{factorial}(3-1)$

↓

$\text{factorial}(2-1)$

↓

$\text{factorial}(1-1) = 1$

✗

選択（ウ）

$\text{factorial}(4) = 4$

✗

選択（エ）

$\text{factorial}(4) = 4 \times (4 - 1)$

✗

選択（オ）

$\text{factorial}(4) = 4 \times \text{factorial}(1)$

↓

$1 \times \text{factorial}(1)$

↓

$1 \times \text{factorial}(1)$

⋮

✗

選択（カ）

$\text{factorial}(4) = 4 \times \text{factorial}(4-1)$

↓

$3 \times \text{factorial}(3-1)$

↓

$2 \times \text{factorial}(2-1)$

↓

$1 \times \text{factorial}(1-1)$

↓

1

◆13 問目/26 問 (問題)

問 8 次の記述中の に入れる正しい答えを、解答群の中から選べ。

優先度付きキューを操作するプログラムである。優先度付きキューとは扱う要素に優先度を付けたキューであり、要素を取り出す際には優先度の高いものから順番に取り出される。クラス PrioQueue は優先度付きキューを表すクラスである。クラス PrioQueue の説明を図に示す。ここで、優先度は整数型の値 1, 2, 3 のいずれかであり、小さい値ほど優先度が高いものとする。

手続 prioSched を呼び出したとき、出力は の順となる。

コンストラクタ		説明
PrioQueue()		空の優先度付きキューを生成する。

メソッド	戻り値	説明
enqueue(文字列型: s, 整数型: prio)	なし	優先度付きキューに、文字列 s を要素として、優先度 prio で追加する。
dequeue()	文字列型	優先度付きキューからキュー内で最も優先度の高い要素を取り出して返す。最も優先度の高い要素が複数あるときは、そのうちの最初に追加された要素の一つを取り出して返す。
size()	整数型	優先度付きキューに格納されている要素の個数を返す。

図 クラス PrioQueue の説明

【プログラム】

```
○prioSched()  
  PrioQueue: prioQueue ← PrioQueue()  
  prioQueue.enqueue("A", 1)  
  prioQueue.enqueue("B", 2)  
  prioQueue.enqueue("C", 2)  
  prioQueue.enqueue("D", 3)  
  prioQueue.dequeue() /* 戻り値は使用しない */  
  prioQueue.dequeue() /* 戻り値は使用しない */  
  prioQueue.enqueue("D", 3)  
  prioQueue.enqueue("B", 2)  
  prioQueue.dequeue() /* 戻り値は使用しない */  
  prioQueue.dequeue() /* 戻り値は使用しない */  
  prioQueue.enqueue("C", 2)  
  prioQueue.enqueue("A", 1)  
  while (prioQueue.size() が 0 と等しくない)  
    prioQueue.dequeue() の戻り値を出力  
  endwhile
```

解答群

ア	"A", "B", "C", "D"
イ	"A", "B", "D", "D"
ウ	"A", "C", "C", "D"
エ	"A", "C", "D", "D"

<  回答用紙 >

◆13 問目/26 問（解説メモ用）

要素に優先度が付与されており、優先度が高い（優先度の数値が小さい）データから順に取り出される



【クラスの構成要素】メンバ変数・メソッド・コンストラクタ

コンストラクタ		説明
PrioQueue()		空の優先度付きキューを生成する。
メソッド	戻り値	説明
enqueue(文字列型: s, 整数型: prio)	なし	優先度付きキューに、文字列 s を要素として、優先度 prio で追加する。
dequeue()	文字列型	優先度付きキューからキュー内で最も優先度の高い要素を取り出して返す。最も優先度の高い要素が複数あるときは、そのうちの最初に追加された要素の一つを取り出して返す。
size()	整数型	優先度付きキューに格納されている要素の個数を返す。

→ インスタンス生成と同時にキューを作成

→ 要素s とその優先度 prio をキューに格納

→ キューの中で最も優先度の高い要素を取り出す
（優先度が等しい場合は入った順に取り出す）

→ キューの中の要素の個数を返す

図 クラス PrioQueue の説明

【プログラム】

```
OprioSched()
PrioQueue: prioQueue ← PrioQueue()
prioQueue.enqueue("A", 1)
prioQueue.enqueue("B", 2)
prioQueue.enqueue("C", 2)
prioQueue.enqueue("D", 3)
prioQueue.dequeue() /* 戻り値は使用しない */
prioQueue.dequeue() /* 戻り値は使用しない */
prioQueue.enqueue("D", 3)
prioQueue.enqueue("B", 2)
prioQueue.dequeue() /* 戻り値は使用しない */
prioQueue.dequeue() /* 戻り値は使用しない */
prioQueue.enqueue("C", 2)
prioQueue.enqueue("A", 1)
while (prioQueue.size() が 0 と等しくない)
    prioQueue.dequeue() の戻り値を出力
endwhile
```



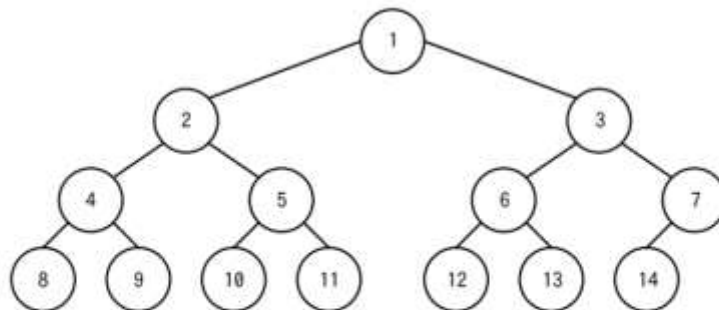
インスタンス
• enqueue
• dequeue
• size

◆14 問目/26 問 (問題)

問9 次の記述中の に入れる正しい答えを、解答群の中から選べ。ここで、配列の要素番号は1から始まる。

手続 order は、図の 2 分木の、引数で指定した節を根とする部分木をたどりながら、全ての節番号を出力する。大域の配列 tree が図の 2 分木を表している。配列 tree の要素は、対応する節の子の節番号を、左の子、右の子の順に格納した配列である。例えば、配列 tree の要素番号 1 の要素は、節番号 1 の子の節番号から成る配列であり、左の子の節番号 2、右の子の節番号 3 を配列 {2, 3} として格納する。

手続 order を order(1)として呼び出すと、 の順に出力される。



注記1 ○の中の値は節番号である。

注記2 子の節が一つの場合は、左の子の節とする。

図 プログラムが扱う 2 分木

【プログラム】

大域：整数型配列の配列：tree ← {{2, 3}, {4, 5}, {6, 7}, {8, 9},
{10, 11}, {12, 13}, {14}, {}, {}, {},
{}, {}, {}, {} // {}は要素数0の配列

```

Order(整数型: n)
  if (tree[n]の要素数 が 2 と等しい)
    order(tree[n][1])
    nを出力
    order(tree[n][2])
  elseif (tree[n]の要素数 が 1 と等しい)
    order(tree[n][1])
    nを出力
  else
    nを出力
  endif
  
```

解答群

- ア 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14
- イ 1, 2, 4, 8, 9, 5, 10, 11, 3, 6, 12, 13, 7, 14
- ウ 8, 4, 9, 2, 10, 5, 11, 1, 12, 6, 13, 3, 14, 7
- エ 8, 9, 4, 10, 11, 5, 2, 12, 13, 6, 14, 7, 3, 1

<  回答用紙 >

要素番号と節の番号が対応しており、子の節番号を左,右の順で格納

大域: 整数型配列の配列: $\text{tree} \leftarrow \{ \{2, 3\}, \{4, 5\}, \{6, 7\}, \{8, 9\},$
 $\{10, 11\}, \{12, 13\}, \{14\}, \{\}, \{\}, \{\},$
 $\{\}, \{\}, \{\}, \{\} \}$ // $\{\}$ は要素数0の配列

```

if (tree[n]の要素数 が 2 と等しい)
    order(tree[n][1])
    nを出力
    order(tree[n][2])
elseif (tree[n]の要素数 が 1 と等しい)
    order(tree[n][1])
    nを出力
else
    nを出力
endif

```

1	2	3	4	5	6	7	8	9
2,3	4,5	6,7	8,9	10,11	12,13	14		

```
tree[n][1]    tree[n][2]
```

1	2	3	4	5	6	7	8	9	10	11	12	13	14
2,3	4,5	6,7	8,9	10,11	12,13	14							

[illegible]

◆15 問目/26 問 (問題)

問10 次のプログラム中の に入れる正しい答えを、解答群の中から選べ。

手続 delNode は、単方向リストから、引数 pos で指定された位置の要素を削除する手続である。引数 pos は、リストの要素数以下の正の整数とする。リストの先頭の位置を 1 とする。

クラス ListElement は、単方向リストの要素を表す。クラス ListElement のメンバ変数の説明を表に示す。ListElement 型の変数はクラス ListElement のインスタンスの参照を格納するものとする。大域変数 listHead には、リストの先頭要素の参照があらかじめ格納されている。

表 クラス ListElement のメンバ変数の説明

メンバ変数	型	説明
val	文字型	要素の値
next	ListElement	次の要素の参照 次の要素がないときの状態は未定義

〔プログラム〕

大域: ListElement: listHead // リストの先頭要素が格納されている

```
OdelNode(整数型: pos) /* posは、リストの要素数以下の正の整数 */
ListElement: prev
整数型: i
if (pos が 1 と等しい)
    listHead ← listHead.next
else
    prev ← listHead
    /* posが2と等しいときは繰返し処理を実行しない */
    for (i を 2 から pos - 1 まで 1 ずつ増やす)
        prev ← prev.next
    endfor
    prev.next ← 
endif
```

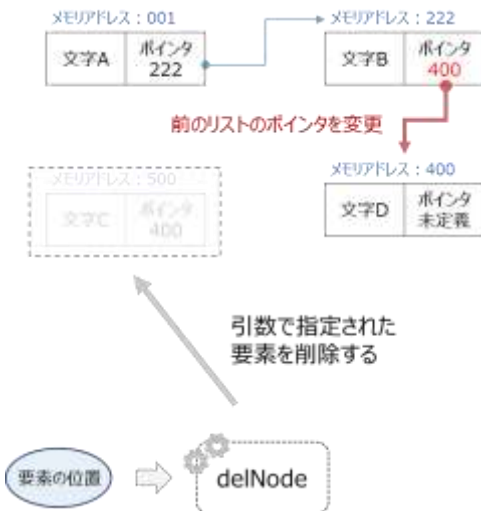
解答群

- | | |
|----------------------|------------------|
| ア listHead | イ listHead.next |
| ウ listHead.next.next | エ prev |
| オ prev.next | カ prev.next.next |

引用：基本情報技術者試験 B試験サンプル問題 問10

< 📄 回答用紙 >

◆15 問目/26 問（解説メモ用）



【プログラム】

大域: ListElement: listHead // リストの先頭要素が格納されている

```

OdelNode(整数型: pos) /* posは、リストの要素数以下の正の整数 */
ListElement: prev
整数型: i
if (pos が 1 と等しい)
    listHead ← listHead.next
else
    prev ← listHead
    /* posが2と等しいときは繰り返し処理を実行しない */
    for (i を 2 から pos - 1 まで 1 ずつ増やす)
        prev ← prev.next
    endfor
    prev.next ← 
endif

```

引用: 基本情報技術者試験 B試験サンプル問題 問10e

先頭要素を削除する処理
2番目の要素のメモリアドレスを先頭リストのメモリアドレスにする

先頭以外の要素を削除する処理

<例> pos=4 のときの処理



【プログラム】

大域: ListElement: listHead // リストの先頭要素が格納されている

```

OdelNode(整数型: pos) /* posは、リストの要素数以下の正の整数 */
ListElement: prev
整数型: i
if (pos が 1 と等しい)
    listHead ← listHead.next
else
    prev ← listHead
    /* posが2と等しいときは繰り返し処理を実行しない */
    for (i を 2 から pos - 1 まで 1 ずつ増やす)
        prev ← prev.next
    endfor
    prev.next ← 
endif

```

削除する要素の1つ前のポインタ
(青い箱) に何を入れるか考える
→ 次の次のリストのメモリアドレスを入れたい

引用: 基本情報技術者試験 B試験サンプル問題 問10

解答群

ア listHead
ウ listHead.next.next
オ prev.next
イ listHead.next
エ prev
カ prev.next.next

先頭要素を削除する処理
2番目の要素のメモリアドレスを先頭リストのメモリアドレスにする

先頭以外の要素を削除する処理

<例> pos=4 のときの処理

for文は削除したい要素の1つ前の要素のメモリアドレスをprevに格納

prev : 009



◆11～15 問目 まとめ（気づきやミスを追加してメモしておきましょう！）

問題	タイプ	問題概要	ポイント（気づき/解法/得たスキルなど）
11	穴埋め	シフト演算を活用してビットの並びを逆にするプログラムを作成	プログラムの情報が少ない場合、選択肢の処理を1つ1つ確認して回答する（時間がかかりそうな場合、本試験では飛ばして良い）
12	穴埋め	再起処理を用いて階乗を返すプログラムを作成	処理が単純な場合、具体的な数値を用意して総当たりで選択肢を確認して回答する
13	処理結果	キューを操作するプログラムの実行結果	- メソッドの参照処理が大半を占めており、一見難しそうだが処理はシンプル - プログラム実行後のキューの結果を見たいので、実際のキューを用意して操作する
14	処理結果	再起処理を用いて2分木の節を出力するプログラムの結果	- プログラムが動画で説明した疑似言語の文法と100%一致していなくても、問題文及びアルゴリズムの知識からある程度の予測を立てて対応する - 再起処理のトレースではプログラムも追跡する
15	穴埋め	オブジェクト指向を用いた単方向リストの要素削除処理を行うプログラムを作成	慣れてきたり、既視感のあるプログラムでは、1行ずつトレースせず、プログラムが意図する処理を予測して穴埋めすることで時短になる（予測難しい場合は、1行ずつ確実にトレースを行う）

5.4. 16～21 問目

◆16 問目/26 問 (問題)

問11 次の記述中の に入れる正しい答えを、解答群の中から選べ。ここで、配列の要素番号は 1 から始まる。

関数 binSort を binSort() として呼び出すと、戻り値の配列には未定義の要素は含まれておらず、値は昇順に並んでいる。

〔プログラム〕

```
○整数型の配列: binSort(整数型の配列: data)
  整数型: n ← dataの要素数
  整数型の配列: bins ← {n個の未定義の値}
  整数型: i

  for (i を 1 から n まで 1 ずつ増やす)
    bins[data[i]] ← data[i]
  endfor

  return bins
```

解答群

- | | |
|----------------------|----------------------|
| ア {2, 6, 3, 1, 4, 5} | イ {3, 1, 4, 4, 5, 2} |
| ウ {4, 2, 1, 5, 6, 2} | エ {5, 3, 4, 3, 2, 6} |

引用：基本情報技術者試験 B試験サンプル問題 問11

<  回答用紙 >

◆16 問目/26 問（解説メモ用）

【プログラム】

```
○整数型の配列: binSort(整数型の配列: data)
  整数型: n ← dataの要素数
  整数型の配列: bins ← {n個の未定義の値}
  整数型: i

  for (i を 1 から n まで 1 ずつ増やす)
    bins[data[i]] ← data[i]
  endfor
  i = 1 ~ 6 で繰り返し

  return bins
```

解答群

- ア {2, 6, 3, 1, 4, 5} イ {3, 1, 4, 4, 5, 2}
ウ {4, 2, 1, 5, 6, 2} エ {5, 3, 4, 3, 2, 6}

引用：基本情報技術者試験 B試験サンプル問題 問11

選択肢 (ア)～(エ) を引数とした際の処理結果を1つずつ求める

配列data					
1	2	3	4	5	6
2	6	3	1	4	5

配列bins					
1	2	3	4	5	6

◆17 問目/26 問 (問題)

問12 次のプログラム中の に入れる正しい答えを、解答群の中から選べ。ここで、配列の要素番号は 1 から始まる。

関数 `simRatio` は、引数として与えられた要素数 1 以上の二つの文字型の配列 `s1` と `s2` を比較し、要素数が等しい場合は、配列の並びがどの程度似ているかの指標として、(要素番号が同じ要素の文字同士が一致する要素の組みの個数 ÷ `s1` の要素数) を実数型で返す。例えば、配列の全ての要素が一致する場合の戻り値は 1、いずれの要素も一致しない場合の戻り値は 0 である。

なお、二つの配列の要素数が等しくない場合は、-1 を返す。

関数 `simRatio` に与える `s1`, `s2` 及び戻り値の例を表に示す。プログラムでは、配列の領域外を参照してはならないものとする。

表 関数 `simRatio` に与える `s1`, `s2` 及び戻り値の例

s1	s2	戻り値
{"a", "p", "p", "l", "e"}	{"a", "p", "p", "l", "e"}	1
{"a", "p", "p", "l", "e"}	{"a", "p", "r", "i", "l"}	0.4
{"a", "p", "p", "l", "e"}	{"n", "e", "l", "o", "n"}	0
{"a", "p", "p", "l", "e"}	{"p", "e", "n"}	-1

[プログラム]

```
○実数型: simRatio(文字型の配列: s1, 文字型の配列: s2)
  整数型: i, cnt ← 0
  if (s1の要素数 ≠ s2の要素数)
    return -1
  endif
  for (i を 1 から s1の要素数 まで 1 ずつ増やす)
    if (  )
      cnt ← cnt + 1
    endif
  endfor
  return cnt ÷ s1の要素数 /* 実数として計算する */
```

解答群

- | | |
|--------------------------------|------------------------------|
| ア <code>s1[i] ≠ s2[cnt]</code> | イ <code>s1[i] ≠ s2[i]</code> |
| ウ <code>s1[i] = s2[cnt]</code> | エ <code>s1[i] = s2[i]</code> |

引用：基本情報技術者試験 B試験サンプル問題 問12

< 📄 回答用紙 >

◆17 問目/26 問（解説メモ用）

問12 次のプログラム中の に入れる正しい答えを、解答群の中から選べ。ここで、配列の要素番号は1から始まる。

関数 `simRatio` は、引数として与えられた要素数1以上の二つの文字型の配列 `s1` と `s2` を比較し、要素数が等しい場合は、配列の並びがどの程度似ているかの指標として、(要素番号が同じ要素の文字同士が一致する要素の総数 ÷ `s1` の要素数) を実数型で返す。例えば、配列の全ての要素が一致する場合の戻り値は1、いずれの要素も一致しない場合の戻り値は0である。

なお、二つの配列の要素数が等しくない場合は、-1を返す。

関数 `simRatio` に与える `s1`, `s2` 及び戻り値の例を表に示す。プログラムでは、配列の領域外を参照してはならないものとする。

表 関数 `simRatio` に与える `s1`, `s2` 及び戻り値の例

s1	s2	戻り値
["a", "p", "p", "i", "a"]	["a", "a", "p", "i", "a"]	1
["a", "p", "p", "i", "a"]	["a", "p", "i", "i", "i"]	0.4
["a", "p", "p", "i", "a"]	["a", "a", "i", "a", "a"]	0
["a", "p", "p", "i", "a"]	["p", "a", "a"]	-1

<類似の指標の計算方法>

`s1`の要素数: 5個

同じ要素番号の `s1`と`s2`で一致している要素の数: 2個 (a,p)

→ 類似度: $2 \div 5 = 0.4$



if 文を使用し、配列`s1`,`s2`の要素数で条件分岐と予想

○実数型: `simRatio`(文字型の配列: `s1`, 文字型の配列: `s2`)

整数型: `i`, `cnt` ← 0

if (`s1`の要素数 ≠ `s2`の要素数)

return -1

endif

for (`i` を 1 から `s1`の要素数 まで 1 ずつ増やす)

if ()

cnt ← cnt + 1

endif

endfor

return cnt ÷ `s1`の要素数 /* 実数として計算する */

→ `s1` と `s2` の要素数が等しくない場合に-1を出力

→ `s1` と `s2` の要素数が等しい場合に類似度を出力

(同じ要素番で文字が一致している`s1`と`s2`の要素の組み合わせの数) ÷ (`s1`の要素数)

`cnt` は 同じ要素番で文字が一致している`s1`と`s2`の要素の組み合わせの数

【プログラム】

○実数型: `simRatio`(文字型の配列: `s1`, 文字型の配列: `s2`)

整数型: `i`, `cnt` ← 0

if (`s1`の要素数 ≠ `s2`の要素数)

return -1

endif

for (`i` を 1 から `s1`の要素数 まで 1 ずつ増やす)

if ()

cnt ← cnt + 1

endif

endfor

return cnt ÷ `s1`の要素数 /* 実数として計算する */

解答群

ア `s1[i] ≠ s2[cnt]`

イ `s1[i] ≠ s2[i]`

ウ `s1[i] = s2[cnt]`

エ `s1[i] = s2[i]`

条件を満たすときに `cnt` の数を1増やす処理

→ 条件は “同じ要素番号の配列 `s1` と `s2` の要素が一致”

◆18 問目/26 問 (問題)

問13 次の記述中の に入れる正しい答えを、解答群の中から選べ。ここで、配列の要素番号は 1 から始まる。

関数 search は、引数 data で指定された配列に、引数 target で指定された値が含まれていればその要素番号を返し、含まれていなければ -1 を返す。data は昇順に整列されており、値に重複はない。

関数 search には不具合がある。例えば、data の 場合は、無限ループになる。

[プログラム]

○整数型: search(整数型の配列: data, 整数型: target)

整数型: low, high, middle

low ← 1

high ← dataの要素数

while (low ≤ high)

middle ← (low + high) ÷ 2 の商

if (data[middle] < target)

low ← middle

elseif (data[middle] > target)

high ← middle

else

return middle

endif

endwhile

return -1

解答群

ア 要素数が 1 で、target がその要素の値と等しい

イ 要素数が 2 で、target が data の先頭要素の値と等しい

ウ 要素数が 2 で、target が data の末尾要素の値と等しい

エ 要素に -1 が含まれている

引用：基本情報技術者試験 B試験サンプル問題 問13

< 📄 回答用紙 >

◆18 問目/26 問（解説メモ用）

問13 次の記述中の に入れる正しい答えを、解答群の中から選べ。ここで、配列の要素番号は 1 から始まる。

関数 search は、引数 data で指定された配列に、引数 target で指定された値が含まれていればその要素番号を返し、含まれていなければ -1 を返す。data は昇順に整列されており、値に重複はない。

関数 search には不具合がある。例えば、data の 場合は、無限ループになる。 ⇒引数を選択肢の条件にして1つつ処理を実行させてみる



〔プログラム〕

○整数型: search(整数型の配列: data, 整数型: target)
整数型: low, high, middle

```
low ← 1
high ← dataの要素数
```

```
while (low ≤ high)
  middle ← (low + high) ÷ 2 の商
  if (data[middle] < target)
    low ← middle
  elseif (data[middle] > target)
    high ← middle
  else
    return middle
  endif
endwhile
```

```
return -1
```

(ア) dataの要素数が1で、targetがその要素の値と等しい

配列data	変数 target
1 2	2

(イ) dataの要素数が2で、targetがdataの先頭要素の値と等しい

配列data	変数 target
1 2 2 3	2

(ウ) dataの要素数が2で、targetがdataの末尾要素の値と等しい

配列data	変数 target
1 2 2 3	3

◆19 問目/26 問 (問題)

問14 次の記述中の に入れる正しい答えを、解答群の中から選べ。ここで、配列の要素番号は 1 から始まる。

要素数が 1 以上で、昇順に整列済みの配列を基に、配列を特徴づける五つの値を返すプログラムである。

関数 summarize を `summarize({0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1})` として呼び出すと、戻り値は である。

[プログラム]

```
○実数型: findRank(実数型の配列: sortedData, 実数型: p)
  整数型: i
  i ← (p × (sortedDataの要素数 - 1)) の小数点以下を切り上げた値
  return sortedData[i + 1]

○実数型の配列: summarize(実数型の配列: sortedData)
  実数型の配列: rankData ← {} /* 要素数0の配列 */
  実数型の配列: p ← {0, 0.25, 0.5, 0.75, 1}
  整数型: i
  for (i を 1 から pの要素数 まで 1 ずつ増やす)
    rankDataの末尾 に findRank(sortedData, p[i])の戻り値 を追加する
  endfor
  return rankData
```

解答群

- ア {0.1, 0.3, 0.5, 0.7, 1}
- イ {0.1, 0.3, 0.5, 0.8, 1}
- ウ {0.1, 0.3, 0.6, 0.7, 1}
- エ {0.1, 0.3, 0.6, 0.8, 1}
- オ {0.1, 0.4, 0.5, 0.7, 1}
- カ {0.1, 0.4, 0.5, 0.8, 1}
- キ {0.1, 0.4, 0.6, 0.7, 1}
- ク {0.1, 0.4, 0.6, 0.8, 1}

< 📄 回答用紙 >

◆19 問目/26 問（解説メモ用）

★プログラムから入力・出力を整理する

○実数型: findRank(実数型の配列: sortedData, 実数型: p)

整数型: i

$i \leftarrow (p \times (\text{sortedDataの要素数} - 1))$ の小数点以下を切り上げた値

return sortedData[i + 1]



○実数型の配列: summarize(実数型の配列: sortedData)

実数型の配列: rankData $\leftarrow \{\}$ /* 要素数0の配列 */

実数型の配列: p $\leftarrow \{0, 0.25, 0.5, 0.75, 1\}$

整数型: i

for (i を 1 から pの要素数 まで 1 ずつ増やす)

rankDataの末尾 に findRank(sortedData, p[i])の戻り値 を追加する

endfor

return rankData



【プログラム】

○実数型: findRank(実数型の配列: sortedData, 実数型: p)

整数型: i

$i \leftarrow (p \times (\text{sortedDataの要素数} - 1))$ の小数点以下を切り上げた値

return sortedData[i + 1]

○実数型の配列: summarize(実数型の配列: sortedData)

実数型の配列: rankData $\leftarrow \{\}$ /* 要素数0の配列 */

実数型の配列: p $\leftarrow \{0, 0.25, 0.5, 0.75, 1\}$

整数型: i

for (i を 1 から pの要素数 まで 1 ずつ増やす) i = 1 ~5 で繰り返し

rankDataの末尾 に findRank(sortedData, p[i])の戻り値 を追加する

endfor

return rankData

【プログラム】

○実数型: findRank(実数型の配列: sortedData, 実数型: p)

整数型: i

$i \leftarrow (p \times (\text{sortedDataの要素数} - 1))$ の小数点以下を切り上げた値

return sortedData[i + 1]

配列 sortedData

1	2	3	4	5	6	7	8	9	10
0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9	1

配列 rankData

配列 p

1	2	3	4	5
0	0.25	0.5	0.75	1

◆20 問目/26 問 (問題)

問15 次の記述中の と に入れる正しい答えの組合せを、解答群の中から選べ。

三目並べにおいて自分が勝利する可能性が最も高い手を決定する。次の手順で、ゲームの状態遷移を木構造として表現し、根以外の各節の評価値を求める。その結果、根の子の中で最も評価値が高い手を、最も勝利する可能性が高い手とする。自分が選択した手を○で表し、相手が選択した手を×で表す。

【手順】

- (1) 現在の盤面の状態を根とし、勝敗がつくか、引き分けとなるまでの考えられる全ての手を木構造で表現する。
- (2) 葉の状態を次のように評価する。
 - ① 自分が勝ちの場合は 10
 - ② 自分が負けの場合は -10
 - ③ 引き分けの場合は 0
- (3) 葉以外の節の評価値は、その節全ての子の子の評価値を基に決定する。
 - ① 自分の手番の節である場合、子の子の評価値で最大の評価値を節の評価値とする。
 - ② 相手の手番の節である場合、子の子の評価値で最小の評価値を節の評価値とする。

ゲームが図の最上部にある根の状態のとき、自分が選択できる手は三つある。そのうち A が指す子の子の評価値は であり、B が指す子の子の評価値は である。

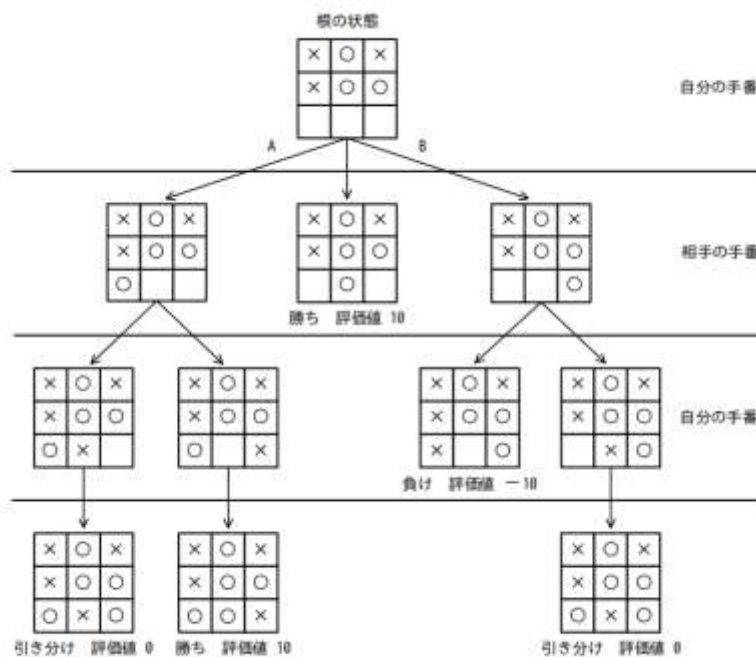


図 三目並べの状態遷移

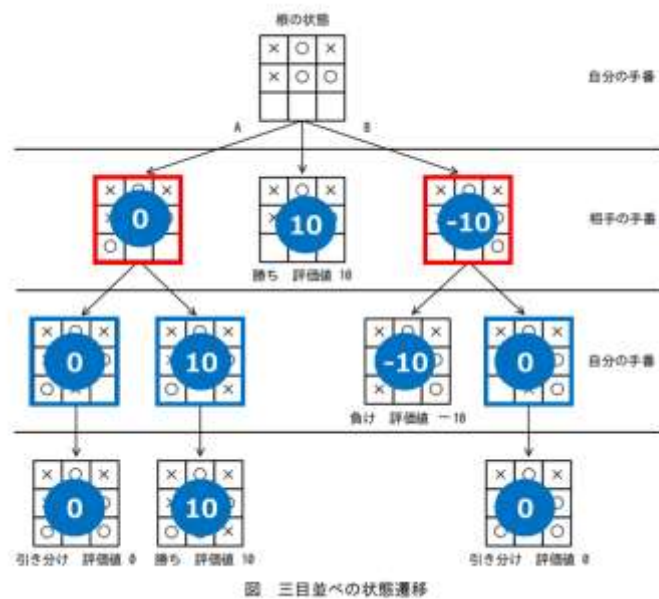
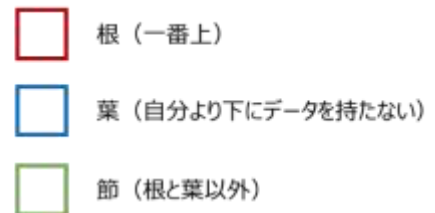
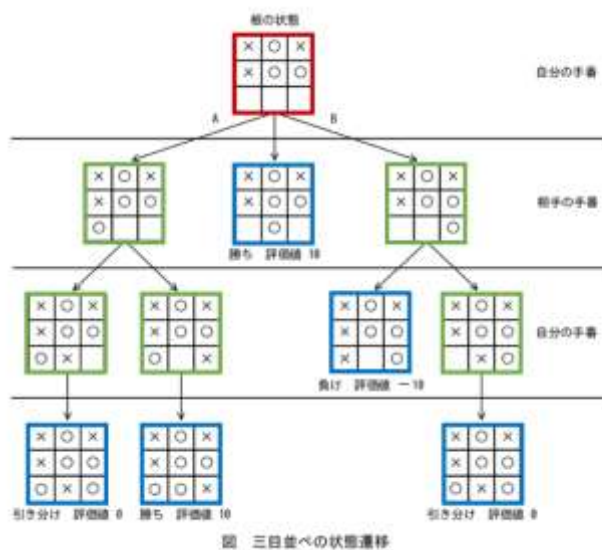
解答群

	a	b
ア	0	-10
イ	0	0
ウ	10	-10
エ	10	0

引用：基本情報技術者試験 B試験サンプル問題 問15

<  回答用紙 >

◆20 問目/26 問（解説メモ用）



子の評価値の最小値が節の評価値

子の評価値の最大値が節の評価値

◆21 問目/26 問 (問題)

問16 次のプログラム中の に入れる正しい答えを、解答群の中から選べ。二つの には、同じ答えが入る。ここで、配列の要素番号は 1 から始まる。

Unicode の符号位置を、UTF-8 の符号に変換するプログラムである。本問で数値の後ろに“(16)”と記載した場合は、その数値が 16 進数であることを表す。

Unicode の各文字には、符号位置と呼ばれる整数値が与えられている。UTF-8 は、Unicode の文字を符号化する方式の一つであり、符号位置が 800(16) 以上 FFFF(16) 以下の文字は、次のように 3 バイトの値に符号化する。

3 バイトの長さのビットパターンを $1110\underline{x}xxx$ $10\underline{x}xxxxx$ $10\underline{x}xxxxx$ とする。ビットパターンの下線の付いた“ $\underline{x}$ ”の箇所に、符号位置を 2 進数で表した値を右詰めで格納し、余った“ $\underline{x}$ ”の箇所に、0 を格納する。この 3 バイトの値が UTF-8 の符号である。

例えば、ひらがなの“あ”の符号位置である 3042(16) を 2 進数で表すと 11000001000010 である。これを、上に示したビットパターンの“ $\underline{x}$ ”の箇所に右詰めで格納すると、 $1110\underline{x}x11$ $10\underline{000001}$ $10\underline{000010}$ となる。余った二つの“ $\underline{x}$ ”の箇所に 0 を格納すると、“あ”の UTF-8 の符号 11100011 10000001 10000010 が得られる。

関数 encode は、引数で渡された Unicode の符号位置を UTF-8 の符号に変換し、先頭から順に 1 バイトずつ要素に格納した整数型の配列を返す。encode には、引数として、800(16) 以上 FFFF(16) 以下の整数値だけが渡されるものとする。

[プログラム]

○整数型の配列: encode(整数型: codePoint)

/* utf8Bytesの初期値は、ビットパターンの“ $\underline{x}$ ”を全て0に置き換え、
8桁ごとに区切って、それぞれを2進数とみなしたときの値 */

整数型の配列: utf8Bytes ← {224, 128, 128}

整数型: cp ← codePoint

整数型: i

for (i を utf8Bytesの要素数 から 1 まで 1 ずつ減らす)

utf8Bytes[i] ← utf8Bytes[i] + (cp ÷ の余り)

cp ← cp ÷ の商

endfor

return utf8Bytes

解答群

ア $((4 - i) \times 2)$

イ $(2 \text{ の } (4 - i) \text{ 乗})$

ウ $(2 \text{ の } i \text{ 乗})$

エ $(i \times 2)$

オ 2

カ 6

キ 16

ク 64

ケ 256

引用：基本情報技術者試験 B試験サンプル問題 問16

<  回答用紙 >

◆21 問目/26 問（解説メモ用）

問16 次のプログラム中の に入れる正しい答えを、解答群の中から選べ。二つの には、同じ答えが入る。ここで、配列の要素番号は1から始まる。

Unicode の符号位置を、UTF-8 の符号に変換するプログラムである。本問で数値の後ろに“(16)”と記載した場合は、その数値が16進数であることを表す。

Unicode の各文字には、符号位置と呼ばれる整数値が与えられている。UTF-8 は、Unicode の文字を符号化する方式の一つであり、符号位置が 800(16) 以上 FFFF(16) 以下の文字は、次のように3バイトの値に符号化する。

3 バイトの長さのビットパターンを 1110xxxx 10xxxxxx 10xxxxxx とする。ビットパターンの下線の付いた“x”の箇所に、符号位置を2進数で表した値を右詰めで格納し、余った“x”の箇所に、0 を格納する。この3バイトの値が UTF-8 の符号である。

例えば、ひらがなの“あ”の符号位置である 3042(16) を2進数で表すと 11000001000010 である。これを、上に示したビットパターンの“x”の箇所に右詰めで格納すると、1110xx11 10000001 10000010 となる。余った二つの“x”の箇所に0を格納すると、“あ”の UTF-8 の符号 11100011 10000001 10000010 が得られる。

関数 encode は、引数で渡された Unicode の符号位置を UTF-8 の符号に変換し、先頭から順に1バイトずつ要素に格納した整数型の配列を返す。encode には、引数として、800(16) 以上 FFFF(16) 以下の整数値だけが渡されるものとする。

Unicode：0と1からなる文字に割り当てられた識別番号（文字コードと呼ばれ、シフトJISコード等他にも種類がある）

- Unicodeには符号位置と呼ばれる整数値がある
- 整数値は3バイトの値に符号化（変換）される
- 1110xxxx 10xxxxxx 10xxxxxx という形式で表現

<例> “あ” の符号位置：3042₍₁₆₎

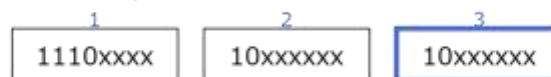
11000001000010 → 引数
1110xxxx 10xxxxxx 10xxxxxx

11100011 10000001 10000010 → 戻り値

〔プログラム〕

```
○整数型の配列: encode(整数型: codePoint)
/* utf8Bytesの初期値は、ビットパターンの“x”を全て0に置き換え、
   8桁ごとに区切って、それぞれを2進数とみなしたときの値 */
整数型の配列: utf8Bytes ← {224, 128, 128}
整数型: cp ← codePoint
整数型: i
for (i を utf8Bytesの要素数 から1まで1ずつ減らす)
  utf8Bytes[i] ← utf8Bytes[i] + (cp ÷  の余り)
  cp ← cp ÷  の商
endfor
return utf8Bytes
```

配列utf8Byte



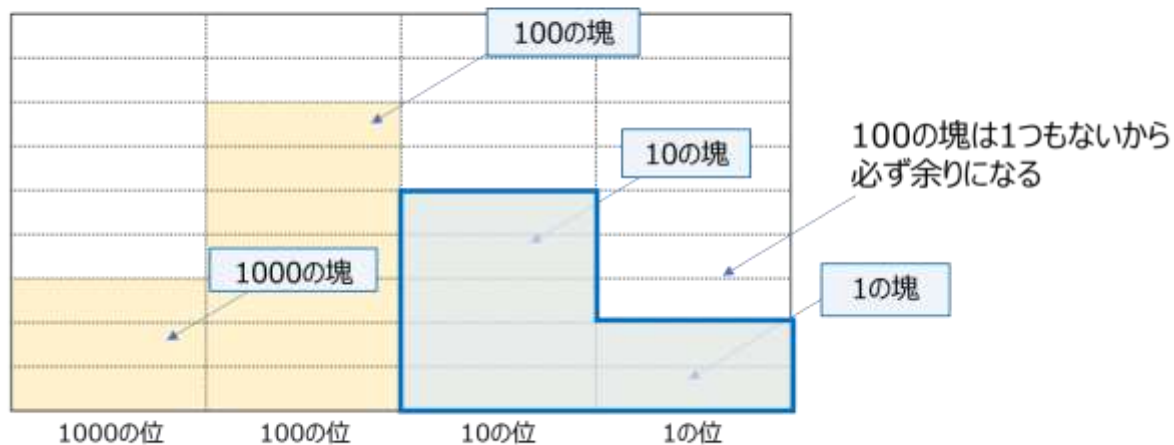
<例> “あ” の符号位置：11000001000010

→ 符号位置の後半6ビットを取り出して3番目の配列に足し算

<例> 3,752から後半2桁を取り出したい

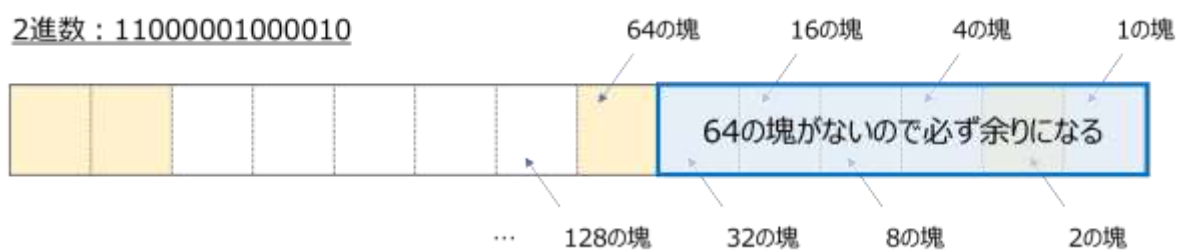
$$3,752 \div 100 = 37 \cdots 52$$

→ 後半2桁を取り出す場合は100で割り算を行う



100で割り算 → 100の塊が何個？

2進数 : 11000001000010



64で割り算 → 64の塊が何個？

64で割り算すると、下位6ビットが余りとして入手

◆16～21 問目 まとめ（気づきやミスを追加してメモしておきましょう！）

問題	タイプ	問題概要	ポイント（気づき/解法/得たスキルなど）
16	-	戻り値の配列に未定義を含まないような引数を求める	選択肢1つずつを引数としてそれぞれプログラムを実行させ、選択肢毎の処理結果を求める
17	穴埋め	文字の類似度を出力するプログラムを作成する	引数や戻り値と問題文の内容から変数の用途を整理する
18	-	無限ループになる引数を求める	while 文の条件が永遠にtrueになる場合、処理の無限ループが起こる
19	処理結果	引数の配列から特定の要素を取り出して出力するプログラムを作成する	問題文から処理の全体像を把握できない場合、プログラムの引数/戻り値を用いて処理を俯瞰する
20	-	三目並べの評価値を求める	問題文から期待される処理を理解し、問題文に従って処理を実行する
21	穴埋め	Unicode の符号位置を UTF-8 の符号に変換するプログラムを作成する	- 応用的な処理をプログラムに落とし込む際は、日本語で記載された処理を正しく整理する - ある数値の下位 x 桁を取り出す場合、$x+1$ 桁の重みで数値を割り算すると、余りが下位 x 桁になる

5.5. 22～26 問目

◆22 問目/26 問 (問題)

問1 次のプログラム中の と に入れる正しい答えの組合せを、
解答群の中から選べ。ここで、配列の要素番号は1から始まる。

関数 findPrimeNumbers は、引数で与えられた整数以下の、全ての素数だけを格納
した配列を返す関数である。ここで、引数に与える整数は2以上である。

[プログラム]

```
○整数型の配列: findPrimeNumbers(整数型: maxNum)
  整数型の配列: pnList ← {} // 要素数0の配列
  整数型: i, j
  論理型: divideFlag
  for (i を 2 から  まで 1 ずつ増やす)
    divideFlag ← true

    /* iの正の平方根の整数部分が2未満のときは、繰返し処理を実行しない */
    for (j を 2 から iの正の平方根の整数部分 まで 1 ずつ増やす) // α
      if (  )
        divideFlag ← false
        αの行から始まる繰返し処理を終了する
      endif
    endfor
    if (divideFlag が true と等しい)
      pnListの末尾 に iの値 を追加する
    endif
  endfor
  return pnList
```

解答群

	a	b
ア	maxNum	i ÷ j の余り が 0 と等しい
イ	maxNum	i ÷ j の商 が 1 と等しくない
ウ	maxNum + 1	i ÷ j の余り が 0 と等しい
エ	maxNum + 1	i ÷ j の商 が 1 と等しくない

引用：基本情報技術者試験 B試験公開問題 問1

<  回答用紙 >

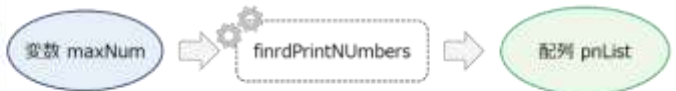
◆22 問目/26 問（解説メモ用）

```

○整数型の配列: findPrimeNumbers(整数型: maxNum)
整数型の配列: pnList ← {} // 要素数0の配列
整数型: i, j
論理型: divideFlag
for (i を 2 から  a まで 1 ずつ増やす)
    divideFlag ← true

    /* i の正の平方根の整数部分が2未満のときは、繰返し処理を実行しない */
    for (j を 2 から i の正の平方根の整数部分 まで 1 ずつ増やす) // α
        if (  b )
            divideFlag ← false
            α の行から始まる繰返し処理を終了する
        endif
    endfor
    if (divideFlag が true と等しい)
        pnListの末尾 に i の値 を追加する
    endif
endfor
return pnList

```



maxNum 以下の素数を pnList に格納

（素数：2以上の自然数で、1と自分自身以外で割り切れる数がない値）

【プログラム】

```

○整数型の配列: findPrimeNumbers(整数型: maxNum)
整数型の配列: pnList ← {} // 要素数0の配列
整数型: i, j
論理型: divideFlag
for (i を 2 から  a まで 1 ずつ増やす)
    ① divideFlag ← true

    ② /* i の正の平方根の整数部分が2未満のときは、繰返し処理を実行しない */
    for (j を 2 から i の正の平方根の整数部分 まで 1 ずつ増やす) // α
        if (  b )
            divideFlag ← false
            α の行から始まる繰返し処理を終了する
        endif
    endfor
    ③ if (divideFlag が true と等しい)
        pnListの末尾 に i の値 を追加する
    endif
endfor
return pnList

```

①～③の処理で、i が素数が判定し、素数の場合 pnList に格納
→ i を 2～ maxNum の間でこの処理を実施

if 文の条件に応じて divideFlagにfalseを格納
→ i が素数が判定する処理

divideFlagが true の時は i を pnListに追加
→ pnList は 素数を格納する配列なので、これは素数を格納する処理
※ divideFlag は i が素数かどうか判定する変数

```

② /* i の正の平方根の整数部分が2未満のときは、繰返し処理を実行しない */
for (j を 2 から i の正の平方根の整数部分 まで 1 ずつ増やす) // α
    if (  b )
        divideFlag ← false
        α の行から始まる繰返し処理を終了する
    endif
endfor

```

i が素数が判定する処理

- divideFlag が trueのときに i が素数
- 空欄b の条件が true のとき divideFlag に false

→ 空欄b は「i が素数でない」という条件

1 と 自分以外で割り切れる数がある

◆23 問目/26 問（問題）

問2 次の記述中の に入れる正しい答えを、解答群の中から選べ。

次のプログラムにおいて、手続 proc2 を呼び出すと、 の順に出力される。

〔プログラム〕

○proc1()
 "A" を出力する
 proc3()

○proc2()
 proc3()
 "B" を出力する
 proc1()

○proc3()
 "C" を出力する

解答群

ア "A", "B", "B", "C"
ウ "A", "C", "B", "C"
オ "B", "C", "B", "A"
キ "C", "B", "A"

イ "A", "C"
エ "B", "A", "B", "C"
カ "C", "B"
ク "C", "B", "A", "C"

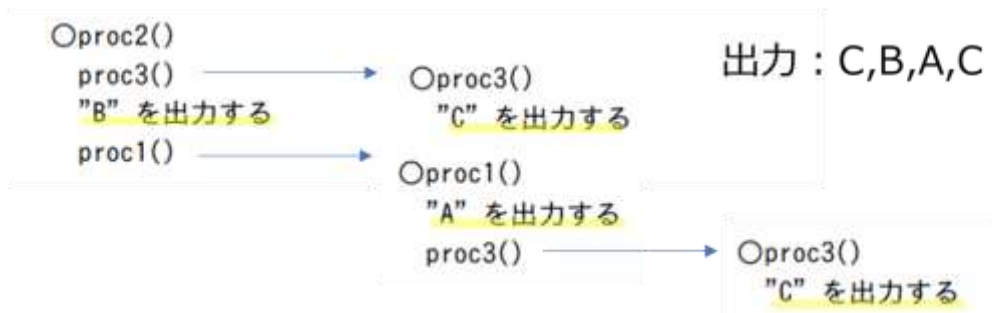
引用：基本情報技術者試験 B試験公開問題 問2

<  回答用紙 >

◆23 問目/26 問（解説メモ用）

問2 次の記述中の に入れる正しい答えを，解答群の中から選べ。

次のプログラムにおいて，手続 proc2 を呼び出すと， の順に出力される。



◆24 問目/26 問 (問題)

問3 次の記述中の に入れる正しい答えを、解答群の中から選べ。ここで、配列の要素番号は1から始まる。

次の手続 sort は、大域の整数型の配列 data の、引数 first で与えられた要素番号から引数 last で与えられた要素番号までの要素を昇順に整列する。ここで、 $first < last$ とする。手続 sort を $sort(1, 5)$ として呼び出すと、`/**  $\alpha$  */` の行を最初に実行したときの出力は “ ” となる。

〔プログラム〕

大域：整数型の配列：data $\leftarrow$ {2, 1, 3, 5, 4}

○sort(整数型：first, 整数型：last)

整数型：pivot, i, j

 pivot $\leftarrow$ data[(first + last) $\div$ 2 の商]

 i $\leftarrow$ first

 j $\leftarrow$ last

while (true)

 while (data[i] < pivot)

 i $\leftarrow$ i + 1

 endwhile

 while (pivot < data[j])

 j $\leftarrow$ j - 1

 endwhile

 if (i $\geq$ j)

 繰返し処理を終了する

 endif

 data[i]とdata[j]の値を入れ替える

 i $\leftarrow$ i + 1

 j $\leftarrow$ j - 1

endwhile

dataの全要素の値を要素番号の順に空白区切りで出力する `/**  $\alpha$  */`

if (first < i - 1)

 sort(first, i - 1)

endif

if (j + 1 < last)

 sort(j + 1, last)

endif

解答群

ア 1 2 3 4 5

イ 1 2 3 5 4

ウ 2 1 3 4 5

エ 2 1 3 5 4

引用：基本情報技術者試験 B試験公開問題 問3

<  回答用紙 >

◆24 問目/26 問 (解説メモ用)

大域: 整数型の配列: $\text{data} \leftarrow \{2, 1, 3, 5, 4\}$

配列 data

1	2	3	4	5
2	1	3	5	4

○sort(整数型: first, 整数型: last)

整数型: pivot, i, j

$\text{pivot} \leftarrow \text{data}[(\text{first} + \text{last}) \div 2 \text{ の商}]$

$i \leftarrow \text{first}$

$j \leftarrow \text{last}$

$\text{pivot} \leftarrow \text{data}[(1 + 5) \div 2] = \text{data}[3] = 3$

$i \leftarrow 1, j \leftarrow 5$

while (true)

while (data[i] < pivot)

① $i \leftarrow i + 1$

endwhile

while (pivot < data[j])

② $j \leftarrow j - 1$

endwhile

if (i ≥ j)

③ 繰返し処理を終了する

endif

④ data[i]とdata[j]の値を入れ替える

$i \leftarrow i + 1$

⑤ $j \leftarrow j - 1$

endwhile

data[i] < pivot の間 i を1増やす

data[3] = 3のとき、条件が false → i = 3

Pivot < data[j] の間 j を1減らす

data[3] = 3のとき、条件が false → j = 3

if 文の条件が true

→ 青いwhile文から抜ける

dataの全要素の値を要素番号の順に空白区切りで出力する /*** α ***/

if (first < i - 1)

sort(first, i - 1)

endif

if (j + 1 < last)

sort(j + 1, last)

endif

◆25 問目/26 問 (問題)

問4 次の記述中の に入れる正しい答えを、解答群の中から選べ。ここで、配列の要素番号は1から始まる。

関数 add は、引数で指定された正の整数 value を大域の整数型の配列 hashArray に格納する。格納できた場合は true を返し、格納できなかった場合は false を返す。ここで、整数 value を hashArray のどの要素に格納すべきかを、関数 calcHash1 及び calcHash2 を利用して決める。

手続 test は、関数 add を呼び出して、hashArray に正の整数を格納する。手続 test の処理が終了した直後の hashArray の内容は、 である。

[プログラム]

大域：整数型の配列：hashArray

```
○論理型: add(整数型: value)
  整数型: i ← calcHash1(value)
  if (hashArray[i] = -1)
    hashArray[i] ← value
    return true
  else
    i ← calcHash2(value)
    if (hashArray[i] = -1)
      hashArray[i] ← value
      return true
    endif
  endif
  return false
```

```
○整数型: calcHash1(整数型: value)
  return (value mod hashArrayの要素数) + 1
```

```
○整数型: calcHash2(整数型: value)
  return ((value + 3) mod hashArrayの要素数) + 1
```

```
○test()
  hashArray ← {5個の -1}
  add(3)
  add(18)
  add(11)
```

解答群

ア $\{-1, 3, -1, 18, 11\}$

イ $\{-1, 11, -1, 3, -1\}$

ウ $\{-1, 11, -1, 18, -1\}$

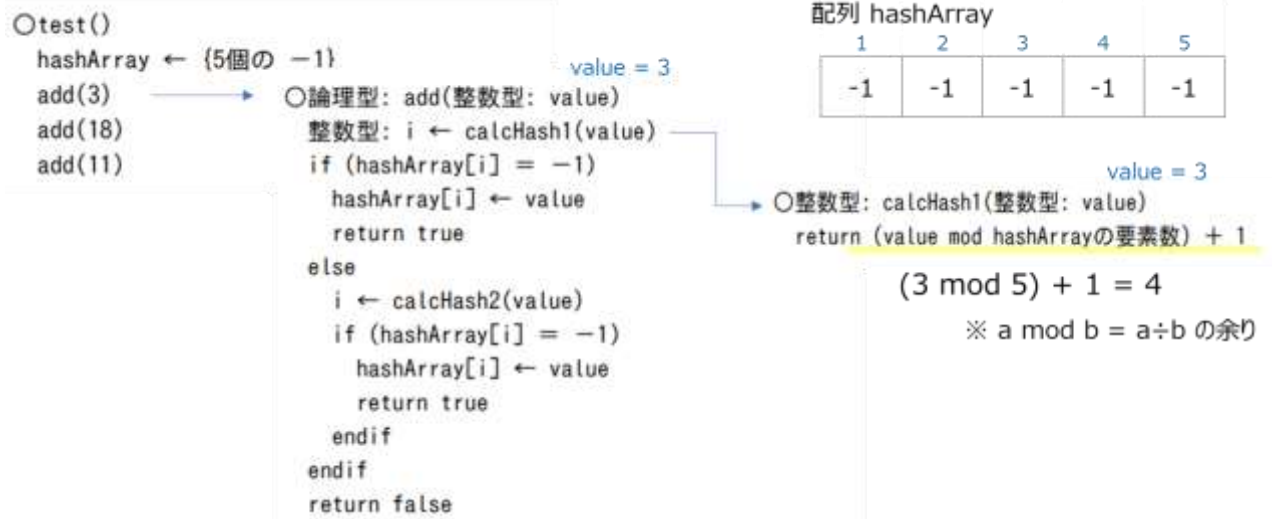
エ $\{-1, 18, -1, 3, 11\}$

オ $\{-1, 18, 11, 3, -1\}$

引用：基本情報技術者試験 B試験公開問題 問4

<  回答用紙 >

◆25 問目/26 問（解説メモ用）



add (18)

- ① $(18 \bmod 5) + 1 = 4 \rightarrow \text{hashArray}[4] \neq -1$
- ③ $((18+3) \bmod 5) + 1 = 2 \rightarrow \text{hashArray}[2] = -1$

add (11)

- ① $(11 \bmod 5) + 1 = 2 \rightarrow \text{hashArray}[2] \neq -1$
- ③ $((11+3) \bmod 5) + 1 = 5 \rightarrow \text{hashArray}[5] = -1$

◆26 問目/26 問 (問題)

問5 次のプログラム中の と に入れる正しい答えの組合せを、解答群の中から選べ。ここで、配列の要素番号は 1 から始まる。

コサイン類似度は、二つのベクトルの向きの類似性を測る尺度である。関数 calcCosineSimilarity は、いずれも要素数が $n(n \geq 1)$ である実数型の配列 vector1 と vector2 を受け取り、二つの配列のコサイン類似度を返す。配列 vector1 が $\{a_1, a_2, \dots, a_n\}$ 、配列 vector2 が $\{b_1, b_2, \dots, b_n\}$ のとき、コサイン類似度は次の数式で計算される。ここで、配列 vector1 と配列 vector2 のいずれも、全ての要素に 0 が格納されていることはないものとする。

$$\frac{a_1b_1 + a_2b_2 + \dots + a_nb_n}{\sqrt{a_1^2 + a_2^2 + \dots + a_n^2} \sqrt{b_1^2 + b_2^2 + \dots + b_n^2}}$$

〔プログラム〕

```
○実数型: calcCosineSimilarity(実数型の配列: vector1,  
                               実数型の配列: vector2)  
実数型: similarity, numerator, denominator, temp ← 0  
整数型: i  
numerator ← 0  
  
for (i を 1 から vector1の要素数 まで 1 ずつ増やす)  
    numerator ← numerator +   
endfor  
  
for (i を 1 から vector1の要素数 まで 1 ずつ増やす)  
    temp ← temp + vector1[i]の2乗  
endfor  
denominator ← tempの正の平方根  
  
temp ← 0  
for (i を 1 から vector2の要素数 まで 1 ずつ増やす)  
    temp ← temp + vector2[i]の2乗  
endfor  
denominator ←   
  
similarity ← numerator ÷ denominator  
return similarity
```

解答群

	a	b
ア	$(\text{vector1}[i] \times \text{vector2}[i])$ の正の平方根	$\text{denominator} \times (\text{tempの正の平方根})$
イ	$(\text{vector1}[i] \times \text{vector2}[i])$ の正の平方根	$\text{denominator} + (\text{tempの正の平方根})$
ウ	$(\text{vector1}[i] \times \text{vector2}[i])$ の正の平方根	tempの正の平方根
エ	$\text{vector1}[i] \times \text{vector2}[i]$	$\text{denominator} \times (\text{tempの正の平方根})$
オ	$\text{vector1}[i] \times \text{vector2}[i]$	$\text{denominator} + (\text{tempの正の平方根})$
カ	$\text{vector1}[i] \times \text{vector2}[i]$	tempの正の平方根
キ	vector1[i]の2乗	$\text{denominator} \times (\text{tempの正の平方根})$
ク	vector1[i]の2乗	$\text{denominator} + (\text{tempの正の平方根})$
ケ	vector1[i]の2乗	tempの正の平方根

引用：基本情報技術者試験 B試験公開問題 問5

< 回答用紙 >

◆26 問目/26 問（解説メモ用）

```

○実数型: calcCosineSimilarity(実数型の配列: vector1,
                             実数型の配列: vector2)
  実数型: similarity, numerator, denominator, temp ← 0
  整数型: i
  numerator ← 0

  for (i を 1 から vector1の要素数 まで 1 ずつ増やす)
    numerator ← numerator + a
  endfor

  for (i を 1 から vector1の要素数 まで 1 ずつ増やす)
    temp ← temp + vector1[i]の2乗
  endfor
  denominator ← tempの正の平方根

  temp ← 0
  for (i を 1 から vector2の要素数 まで 1 ずつ増やす)
    temp ← temp + vector2[i]の2乗
  endfor
  denominator ← b

  similarity ← numerator ÷ denominator
  return similarity

```



similarity : コサイン類似度

$$\frac{a_1b_1 + a_2b_2 + \dots + a_nb_n}{\sqrt{a_1^2 + a_2^2 + \dots + a_n^2} \sqrt{b_1^2 + b_2^2 + \dots + b_n^2}}$$

→ numerator
 → denominator

```

○実数型: calcCosineSimilarity(実数型の配列: vector1,
                             実数型の配列: vector2)
  実数型: similarity, numerator, denominator, temp ← 0
  整数型: i
  numerator ← 0

  for (i を 1 から vector1の要素数 まで 1 ずつ増やす)
    numerator ← numerator + a
  endfor

  for (i を 1 から vector1の要素数 まで 1 ずつ増やす)
    temp ← temp + vector1[i]の2乗
  endfor
  denominator ← tempの正の平方根

  temp ← 0
  for (i を 1 から vector2の要素数 まで 1 ずつ増やす)
    temp ← temp + vector2[i]の2乗
  endfor
  denominator ← b

  similarity ← numerator ÷ denominator
  return similarity

```

$$\frac{a_1b_1 + a_2b_2 + \dots + a_nb_n}{\sqrt{a_1^2 + a_2^2 + \dots + a_n^2} \sqrt{b_1^2 + b_2^2 + \dots + b_n^2}}$$

◆22～26 問目 まとめ（気づきやミスを追加してメモしておきましょう！）

問題	タイプ	問題概要	ポイント（気づき/解法/得たスキルなど）
22	穴埋め	素数を返すプログラムを作成	プログラムを階層化させ、わかりやすい部分から処理の意図を紐どいていく
23	処理結果	あるプログラムの呼び出し結果	特になし
24	処理結果	配列の要素並び替えプログラムの作成	プログラム全体の中で占める割合が小さいwhile文は、トレース表無しで結果だけメモして、プログラムの全体理解を進める（慣れてきたらでOK）
25	処理結果	配列へ要素を格納するプログラムの処理結果	問題文の全体像とプログラムの内容から、処理を予測して一般化させる（慣れてきたらでOK）
26	穴埋め	コサイン類似度を求めるプログラムの作成	問題文及び引数/戻り値を照らし合わせ、変数の用途やプログラムの各階層で実施されている処理の意図を整理する

(参考) 科目 B アルゴリズムの出題範囲

基本情報技術者試験

1 プログラミング全般に関すること

実装するプログラムの要求仕様（入出力、処理、データ構造、アルゴリズムほか）の把握、使用するプログラム言語の仕様に基づくプログラムの実装、既存のプログラムの解読及び変更、処理の流れや変数の変化の想定、プログラムのテスト、処理の誤りの特定（デバッグ）及び修正方法の検討 など

注記 プログラム言語について、基本情報技術者試験では擬似言語を扱う。

2 プログラムの処理の基本要素に関すること

型、変数、配列、代入、算術演算、比較演算、論理演算、選択処理、繰り返し処理、手続・関数の呼出し など

3 データ構造及びアルゴリズムに関すること

再帰、スタック、キュー、木構造、グラフ、連結リスト、整列、文字列処理 など

4 プログラミングの諸分野への適用に関すること

数理・データサイエンス・AI などの分野を題材としたプログラム など

引用：基本情報技術者試験 シラバス